Context-Aware Fine-Tuning of Large Language Models for First-Innings Score Prediction in T20 Cricket

by

Damith Tilanka Abewardana Ranasingha Ratnayaka

University of Sri Jayewardenepura

MSc 2026

Context-Aware Fine-Tuning of Large Language Models for First-Innings Score Prediction in T20 Cricket

by

Damith Tilanka Abewardana Ranasingha Ratnayaka

Thesis submitted to the University of Sri Jayewardenepura

for the award of the Degree of Master of Science

Declaration

The work described in this thesis was carried out by me under the supervision of Dr. Rajitha M. Silva and a report on this has not been submitted in whole or in part to any university or any other institution for another Degree / Diploma.

Name: Damith Tilanka Abewardana Ranasingha Ratnayaka

Student No: MSC/DSA/055

Signature: _____

Date: _____

Supervisor Certification

I / We certify that the above statement made by the candidate is true and that this thesis is suitable for submission to the University for the purpose of evaluation.

Name: Dr. Rajitha M. Silva

Signature of the supervisor: _____

Date: _____

Contents

List of Tables

List of Figures

Abbreviations

API	Application Programming Interface
BBL	Big Bash League
BERT	Bidirectional Encoder Representations from Transformers
CPL	Caribbean Premier League
DLS	Duckworth–Lewis–Stern
GPU	Graphics Processing Unit
ICC	International Cricket Council
IPL	Indian Premier League
JSON	JavaScript Object Notation
LLM	Large Language Model
LoRA	Low-Rank Adaptation
MAE	Mean Absolute Error
MSE	Mean Squared Error
NF4	NormalFloat 4-bit
ODI	One Day International
PEFT	Parameter-Efficient Fine-Tuning
QLoRA	Quantised Low-Rank Adaptation
RBSS	Remaining Batting Strength Score
R^2	Coefficient of Determination
SFT	Supervised Fine-Tuning

T20	Twenty20
TRL	Transformer Reinforcement Learning
VRAM	Video Random Access Memory

Glossary

Batting Strength Score A weighted composite score quantifying the aggregate scoring potential of the batting lineup remaining at a given point in an innings. Each player is assigned a weight based on their classified role: batter (1.0), all-rounder (0.8), utility player (0.65), unknown (0.45), or bowler (0.3).

Checkpoint A saved snapshot of model weights at a specific step during training. In this thesis, checkpoints are saved at regular 100-step intervals and the checkpoint achieving minimum validation loss is selected as the deployment model.

Completion In the context of language model fine-tuning, the target output text that the model is trained to generate given a prompt input. In this work, the completion is the actual final first-innings total expressed as a plain integer string.

Decoder-only Transformer A transformer architecture that generates output tokens autoregressively, conditioning each new token on all preceding tokens. Used in models such as Llama and the GPT series.

Fine-tuning The process of continuing the training of a pre-trained model on a domain-specific dataset to adapt it to a target task, typically using a lower learning rate than the original pre-training.

Greedy Decoding An inference strategy in which the model always selects the token with the highest predicted probability at each generation step, producing deterministic outputs without sampling.

In-game State Snapshot A structured representation of the match state at a specific point during a batting innings, capturing all cumulative and section-level statistics up to that point. In this work, one snapshot is extracted at the end of each two-over section.

Innings In cricket, the period during which one team bats. In the T20 format, each team has one innings of up to 20 overs or until all ten wickets have fallen.

Over A set of six consecutive legal deliveries bowled by a single bowler.

Par Score The historical percentile first-innings total at a given venue, used as a contextual benchmark for typical scoring expectations at that ground. In this work, the 52nd percentile of first-innings totals is used for venues with ten or more historical matches.

Powerplay The first six overs of a T20 innings, during which fielding restrictions are in force. These restrictions typically result in elevated scoring rates relative to the rest of the innings.

Prompt A structured natural-language input provided to a language model to elicit a desired output. In this work, the prompt encodes the full in-game match state in a fixed template format.

QLoRA Quantised Low-Rank Adaptation; a parameter-efficient fine-tuning method that combines 4-bit NormalFloat quantisation of frozen base model weights with trainable low-rank adapter matrices injected into the attention layers.

Run Rate The average number of runs scored per over at a given point in an innings, computed as cumulative runs divided by overs completed.

Section A two-over unit of innings progression used in this work for feature extraction. Each T20 innings is divided into ten non-overlapping sections, and one in-game state snapshot is captured at the end of each section.

Zero-shot Inference Evaluation of a language model on a task without providing any task-specific training examples, relying solely on the knowledge and capabilities encoded during pre-training.

Acknowledgements

I wish to express my sincere and heartfelt gratitude to my research supervisor, Dr. Rajitha M. Silva, for the steadfast support, expert guidance, and constant encouragement extended to me throughout the course of this research and the preparation of this thesis. The thoughtful feedback and wholehearted dedication offered at every stage have been instrumental in shaping the direction and quality of this work.

I am deeply thankful to Dr. Chitraka Wickramarachchi and Dr. Anuradha Ariyaratne, course coordinators of the Master of Data Science and Artificial Intelligence programme, for providing invaluable initial guidance and direction. My appreciation also extends to the Faculty of Graduate Studies of the University of Sri Jayewardenepura for making available the necessary resources and fostering an environment conducive to research.

I am indebted to my colleagues and friends whose willingness to share knowledge and offer constructive feedback has greatly enriched this research experience. Above all, my deepest appreciation goes to my family for their unconditional love, patience, and unwavering encouragement throughout this endeavour.

To everyone who contributed, in whatever capacity, to the realisation of this work — thank you.

Abstract

The rapid proliferation of Twenty20 (T20) cricket as the dominant format of international competition has created a growing demand for accurate, real-time predictive tools that can assess scoring trajectories from incomplete innings data. Despite considerable prior work employing conventional statistical and machine learning methods, the potential of Large Language Models (LLMs) for domain-specific sports forecasting tasks remains largely unexplored. This thesis investigates whether a compact, open-source LLM, enhanced through parameter-efficient fine-tuning, can serve as an effective in-game score predictor for T20 cricket first innings.

A dataset comprising international men's T20 matches sourced from Cricsheet, spanning the period 2005 to 2026, was processed into discrete in-game state snapshots taken at two-over intervals. Each snapshot was characterised by sixteen contextual features, including cumulative runs scored, wickets lost, current run rate, dot ball counts, boundary frequencies, powerplay completion status, and a venue par score derived from historical match outcomes. A novel feature, the *Remaining Batting Strength Score*, was engineered as a weighted composite of the player types yet to bat, providing a quantitative encoding of residual scoring capacity. These structured snapshots were serialised into natural-language prompts and paired with the actual final first-innings total to form a labelled training corpus. A strictly temporal train–validation–test partitioning strategy was adopted to prevent data leakage and ensure evaluation fidelity under realistic deployment conditions.

Three experimental conditions were evaluated on a held-out test set using Mean Absolute Error (MAE), Mean Squared Error (MSE), and the coefficient of determination (R^2) as performance metrics. First, the base *Llama 3.2-3B* model was assessed in a zero-shot setting to establish a pre-training baseline. Second, the same model was subjected to Su-

pervised Fine-Tuning (SFT) using Quantised Low-Rank Adaptation (QLoRA), training exclusively on the domain-specific prompt–completion dataset. Third, the frontier model *GPT-4.1-nano* was evaluated zero-shot to provide an upper-bound reference from a substantially larger, instruction-tuned proprietary system.

The findings of this study demonstrate that domain-specific fine-tuning of a small, resource-efficient LLM yields substantial improvements over the zero-shot baseline and provides a competitive alternative to proprietary frontier models for the task of in-game T20 score prediction. The work contributes a reusable data pipeline, a reproducible fine-tuning framework, and empirical evidence supporting the viability of small fine-tuned LLMs as practical sports analytics tools, particularly in resource-constrained settings.

Chapter 1

Introduction

1.1 Background

Cricket, one of the world's most widely followed team sports, has undergone a transformative evolution with the advent of the Twenty20 (T20) format. Introduced at the international level in 2005, T20 cricket condenses each team's innings to a maximum of twenty overs, producing a high-intensity contest that typically concludes within three hours. The format has experienced extraordinary growth in popularity, with tournaments such as the Indian Premier League (IPL), the ICC Men's T20 World Cup, and the Big Bash League attracting a substantial global broadcast audience. This widespread appeal has simultaneously stimulated a thriving market for sports analytics, where accurate and timely predictions can serve broadcasters, team management, fantasy gaming platforms, and betting markets alike.

At the heart of T20 analytics lies the problem of score prediction: given the current state of a batting team's innings, how many runs are they likely to score in total? The difficulty of this prediction task stems from the high-variance, non-linear dynamics that characterise T20 innings progression: scoring momentum can shift dramatically within a single over. Factors such as the number of wickets in hand, the calibre of batters yet to bat, the characteristics of the venue, the run rate established in the powerplay, and the frequency of boundaries all interact in complex, context-dependent ways. A reliable in-game prediction system must be sensitive to all of these dimensions simultaneously.

1.2 Problem Statement

Traditional approaches to T20 score prediction have largely relied on statistical models and handcrafted regression techniques that capture only a subset of the relevant in-game dynamics. Methods such as the Duckworth–Lewis–Stern (DLS) method, while operationally robust for rain-interrupted matches, were designed to set revised targets rather than to predict absolute first-innings totals from live match states. Machine learning approaches, including linear regression, random forests, and neural networks, have improved predictive accuracy but require careful feature selection and are generally constrained to numeric representations of match state, losing the rich contextual semantics of an evolving game situation.

A fundamental limitation shared by these approaches is that they treat match state as a purely numerical vector. In reality, a cricket analyst communicating match context would naturally describe it in language: *"the batting side has scored 95 runs in 12 overs for the loss of 3 wickets, with their best two batters yet to come, at a ground where the average first-innings total is 165 runs."* This observation motivates the exploration of Large Language Models (LLMs) as an alternative prediction paradigm — one that operates natively over natural-language representations of game state and may benefit from broad world knowledge about teams, venues, and playing conditions acquired during pre-training.

1.3 Research Motivation

The emergence of transformer-based LLMs has produced models with remarkable capabilities across a wide variety of tasks, including question answering, structured prediction, and numerical reasoning. Models such as OpenAI's GPT series and Meta's Llama family have demonstrated that a single pre-trained model, when appropriately prompted or fine-tuned, can adapt to highly specific downstream tasks with minimal additional supervision. However, the application of LLMs to quantitative sports forecasting tasks — particularly in-game prediction under partial observability — remains an underexplored frontier.

Two complementary questions motivate this research. First, can a large, instruction-tuned frontier model leverage its general world knowledge to make competent T20 score predictions from natural-language match-state descriptions in a zero-shot setting? Second, can a substantially smaller, open-source LLM be elevated to comparable or superior performance through domain-specific supervised fine-tuning on a curated dataset of historical T20 innings data? The answers have practical implications: a small fine-tuned model incurs substantially lower inference cost than a proprietary frontier API, making it viable for real-time, resource-constrained applications.

1.4 Research Objectives

The primary objective of this research is to investigate the efficacy of Large Language Models for in-game first-innings score prediction in international men's T20 cricket. The specific objectives are as follows:

1. To construct a comprehensive, temporally ordered dataset of in-game state snapshots from international men's T20 matches, drawn from the Cricsheet repository, and to engineer a set of contextual features — including a novel *Remaining Batting Strength Score* — that encode the scoring potential of the remaining batting lineup.

2. To design a natural-language prompt schema that losslessly encodes each in-game state snapshot into a structured textual representation suitable for LLM input, and to build a labelled training corpus of prompt–completion pairs where the completion is the actual final first-innings total.

3. To establish a zero-shot baseline by evaluating the untuned *Llama 3.2-3B* model on the prediction task, thereby quantifying the extent of domain knowledge present in the pre-trained model without any task-specific supervision.

4. To fine-tune the *Llama 3.2-3B* model on the domain-specific dataset using Supervised Fine-Tuning (SFT) with Quantised Low-Rank Adaptation (QLoRA), and to

evaluate whether fine-tuning yields a measurable and practically significant improvement in prediction accuracy over the zero-shot baseline.

5. To investigate the effect of checkpoint selection on generalisation performance by comparing the minimum validation-loss checkpoint against the final training checkpoint, and to determine which selection strategy produces superior out-of-sample prediction accuracy on the held-out test set.

6. To evaluate the frontier model *GPT-4.1-nano* in a zero-shot setting on the same test set, providing an external reference point from a state-of-the-art proprietary system, and to compare its performance against both the zero-shot and fine-tuned variants of the smaller open-source model.

1.5 Scope and Limitations

This research is scoped to the prediction of first-innings totals in international men's T20 cricket. Only completed first innings — those reaching twenty overs or concluding with ten wickets fallen — are considered, thereby excluding rain-reduced or otherwise interrupted matches. The study does not address second-innings target chasing, live win probability estimation, or player-level performance prediction.

The dataset is restricted to international-level men's T20 matches available through the Cricsheet data repository. Domestic T20 franchise competitions (e.g., the IPL, BBL, or CPL) are not included, which may limit the generalisability of the fine-tuned model to those playing environments. Furthermore, the fine-tuning experiments are conducted using a parameter-efficient adapter-based approach (QLoRA) rather than full-parameter fine-tuning, owing to the computational constraints of the training infrastructure available.

The Remaining Batting Strength Score also relies on historical per-player career statistics derived from the training corpus. Players with insufficient career data — those appearing in fewer than fifteen international matches, or accumulating fewer than 250 runs and 10 wickets — are assigned a generic fallback weight, which may reduce the accuracy of this

feature for matches involving less-established sides or debutant players.

1.6 Thesis Organisation

The remainder of this thesis is structured as follows. Chapter 2 reviews the existing literature on cricket score prediction, sports analytics using machine learning, and the application of Large Language Models to structured prediction tasks. Chapter 3 describes the data collection, pre-processing, and feature engineering pipeline, including the construction of the in-game state dataset, the design of the prompt schema, the fine-tuning procedure, the evaluation framework, and the deployment architecture. Chapter 4 presents the empirical results across all three experimental conditions. Chapter 5 discusses the broader implications of the findings, acknowledges the limitations of the study, and proposes directions for future work. Chapter 6 states the conclusions of the research. Chapter 7 provides recommendations for practitioners and future researchers.

Chapter 2

Literature Review

2.1 Introduction

This chapter reviews the body of knowledge relevant to the present research. The review is organised into five thematic areas. Section 2.2 examines the evolution of cricket score prediction, from early statistical frameworks to contemporary machine learning approaches. Section 2.3 surveys the development of transformer-based Large Language Models and the capabilities that underpin their use in structured prediction tasks. Section 2.4 discusses parameter-efficient fine-tuning methods, with particular attention to Low-Rank Adaptation (LoRA) and its quantised variant, QLoRA. Section 2.5 reviews the emerging literature on applying LLMs to sports analytics and numerical forecasting. Section 2.6 synthesises the reviewed work and identifies the gaps that the present study addresses.

The literature search was conducted using Google Scholar, Semantic Scholar, and arXiv, employing search terms encompassing "T20 cricket prediction", "cricket score forecasting", "large language models sports", "LLM fine-tuning sports analytics", and "transformer cricket prediction", covering publications up to April 2026.

2.2 Cricket Score Prediction

2.2.1 Early Statistical Approaches

Quantitative analysis of cricket outcomes has a long history in the sports science literature. Among the earliest and most influential contributions is the resource-allocation framework developed by Duckworth and Lewis (1998), later revised by Stern (2016) to produce the

Duckworth–Lewis–Stern (DLS) method [10, 30]. While the DLS method was designed principally to set revised targets in rain-interrupted limited-overs matches, it introduced the concept of *resources* — a joint function of overs remaining and wickets in hand — as the fundamental currency of a batting innings. This conceptual framing has influenced much subsequent work, including the present study's treatment of remaining batting capacity as a primary predictive feature.

Subsequent statistical work sought to model the run-scoring process more directly. Perera and Swartz (2012) proposed a Markov chain model of ball-by-ball outcomes, demonstrating that transitions between batting states (runs scored, wickets lost) could be captured by a set of transition probabilities estimated from historical data [25]. Such models offer interpretability but assume stationarity of the transition probabilities across venues, opposition, and match conditions — an assumption that limits their applicability to the heterogeneous contexts of modern international T20 cricket.

2.2.2 Machine Learning Approaches to Score Prediction

The availability of ball-by-ball match data in structured formats, exemplified by repositories such as Cricsheet [6], has enabled a shift towards data-driven machine learning models. Sankaranarayanan et al. (2014) applied regression trees and support vector regression to predict ODI match outcomes, reporting improvements over linear baselines when non-linear feature interactions were exploited [29]. Similar techniques were subsequently applied to T20 data, where the shorter format introduces greater variance in outcomes and makes mid-innings prediction particularly challenging.

Kampakis and Thomas (2015) investigated machine learning classifiers for predicting English county T20 match outcomes, finding that ensemble methods such as random forests outperformed logistic regression, and that features encoding team strength and recent form were among the most discriminative [18]. Their work highlighted the importance of incorporating qualitative team composition information alongside raw scoring statistics, a finding that directly motivates feature engineering approaches based on team composition

and player quality.

Recurrent neural networks (RNNs) and long short-term memory (LSTM) networks have been applied to model the sequential nature of an innings. Predicting final scores from partial ball-by-ball sequences is a natural time-series completion task, and LSTMs can, in principle, capture long-range temporal dependencies within an innings. Iyer and Sharda (2009) were among the first to apply neural networks to cricket performance prediction, while more recent work has applied deep learning to predict over-by-over run tallies [17]. However, LSTM-based approaches require substantial volumes of aligned sequential data and are sensitive to the choice of sequence granularity, making them less straightforward to deploy than snapshot-based regression models.

2.2.3 Feature Engineering in T20 Analytics

A recurring theme in the cricket analytics literature is the importance of thoughtful feature engineering. Beyond raw cumulative statistics, researchers have identified several categories of features with strong predictive value.

Venue characteristics have been consistently shown to exert a significant influence on scoring patterns. Preston and Thomas (2000) demonstrated that the pitch and outfield conditions at different grounds produce systematic differences in run rates that persist across matches and are not fully explained by team-level factors [27]. The present study operationalises this insight through a per-venue par score derived from historical win percentages — a measure of the typical score at which a batting-first team has an approximately equal chance of winning at a given ground.

Batting depth and player quality represent another critical dimension. Bailey and Clarke (2004) showed, in the context of one-day international cricket, that remaining wickets are a stronger predictor of final innings totals than runs scored at a given point in an innings [3]. While the T20 format differs in pace and strategy from the 50-over game, the principle that batting depth constitutes a key resource remains applicable. Extending this, the present study introduces a *Remaining Batting Strength Score* that weights remaining play-

ers according to their classified role (specialist batter, all-rounder, utility player, or bowler), capturing not merely the quantity but the quality of the remaining batting resources.

Powerplay dynamics also play a distinctive role in T20 cricket. The mandatory fielding restrictions in the first six overs create a structurally different run-scoring environment compared to the middle and death overs. Features that explicitly flag powerplay completion therefore allow a model to condition its predictions on the phase of play.

2.2.4 Limitations of Conventional Approaches

A common limitation of the approaches reviewed above is their reliance on numeric feature vectors as inputs. This design choice, while computationally convenient, discards the contextual and semantic richness of match state. A numerical encoding of "overs: 12, runs: 95, wickets: 3" treats all matches with identical numeric profiles identically, regardless of whether the batting team possesses a world-class finisher yet to bat or a tail-end bowler. Language-based representations offer a more expressive medium in which such qualitative distinctions can be communicated naturally.

2.3 Large Language Models

2.3.1 Transformer Architecture

The modern LLM ecosystem traces its origins to the transformer architecture introduced by Vaswani et al. (2017) [32]. The transformer replaces the recurrence of RNNs with a self-attention mechanism, enabling each token in a sequence to attend directly to every other token, irrespective of their relative positions. This architectural choice facilitates highly parallelisable training and has proven to scale effectively with model size, data volume, and compute budget.

The transformer comprises an encoder and a decoder, though subsequent work has demonstrated the utility of encoder-only models (e.g., BERT [9]) for classification and sequence labelling tasks, and decoder-only models (e.g., the GPT family [5]) for generative language

modelling. The present study employs decoder-only architectures, specifically Llama 3.2-3B and GPT-4.1-nano, as these models are trained to predict the next token in a sequence — a capability directly applicable to the completion-style prediction task of generating a final score from a match-state prompt.

2.3.2 Pre-training and the Emergence of General Capabilities

Decoder-only LLMs are pre-trained on vast corpora of text using a next-token prediction objective. Brown et al. (2020) demonstrated with GPT-3 that models trained at sufficient scale exhibit *emergent* capabilities: the ability to perform novel tasks from natural-language instructions or few-shot examples without any gradient-based task-specific adaptation [5]. This phenomenon, termed *in-context learning*, suggests that large pre-trained models acquire a form of general reasoning that can be directed towards structured prediction tasks through careful prompt engineering.

Touvron et al. (2023) introduced LLaMA, demonstrating that carefully curated pre-training data could produce open-source models competitive with larger proprietary systems [31]. The subsequent Llama 2 and Llama 3 model families further improved instruction-following capability and contextual reasoning. Llama 3.2, the specific variant employed in the present study, is a 3-billion-parameter decoder-only model released by Meta AI (2024) [21], trained on a multilingual corpus and made publicly available under a permissive research licence, making it an accessible platform for domain-specific fine-tuning experiments.

2.3.3 LLMs for Numerical and Structured Reasoning

A prominent concern regarding the use of LLMs for quantitative prediction tasks is their inconsistent performance on arithmetic and numerical reasoning. Wei et al. (2022) showed that chain-of-thought prompting substantially improves multi-step mathematical reasoning in large models, though the improvement is less pronounced for smaller models below approximately 100 billion parameters [34]. For the present task — which requires not

multi-step arithmetic but rather a mapping from structured context to a plausible output range — the relevant capability is the model's capacity to encode, through supervised fine-tuning, the statistical regularity between structured match-state descriptions and their associated final score distributions. Fine-tuning on domain data is expected to instil precisely this kind of conditional pattern recognition.

2.4 Parameter-Efficient Fine-Tuning

2.4.1 The Challenge of Full Fine-Tuning

While pre-training equips LLMs with broad general capabilities, adapting them to specific downstream tasks typically requires some form of supervised fine-tuning on task-labelled data. Full fine-tuning — updating all model parameters with gradient descent on the target dataset — is computationally expensive for multi-billion-parameter models and risks catastrophic forgetting of pre-trained knowledge when the fine-tuning corpus is small [19]. These constraints motivate the development of parameter-efficient fine-tuning (PEFT) methods that achieve task adaptation by updating only a small subset of parameters while keeping the pre-trained weights frozen.

2.4.2 Low-Rank Adaptation (LoRA)

Hu et al. (2022) proposed Low-Rank Adaptation (LoRA), a PEFT method that injects trainable low-rank decomposition matrices into the frozen weight matrices of a pre-trained model [14]. Specifically, for a weight matrix $W \in \mathbb{R}^{d\times k}$, LoRA parameterises the update as:

$$W' = W + \Delta W = W + BA \tag{2.1}$$

where $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times k}$ are trainable matrices with rank $r \ll \min(d, k)$. Only A and B are updated during fine-tuning; W is frozen. This approach reduces the number of trainable parameters by a factor of $dk/(r(d + k))$, which for typical transformer atten-

tion weight matrices represents a reduction of two to three orders of magnitude compared to full fine-tuning. Hu et al. demonstrated that LoRA-adapted models match or exceed the performance of fully fine-tuned baselines on a range of language understanding and generation benchmarks, with a fraction of the trainable parameter count.

2.4.3 Quantised Low-Rank Adaptation (QLoRA)

Dettmers et al. (2023) extended LoRA with quantisation to produce QLoRA, enabling fine-tuning of large models on consumer-grade hardware [8]. QLoRA combines three techniques: (i) 4-bit NormalFloat (NF4) quantisation of the frozen pre-trained weights, which exploits the approximately normal distribution of pre-trained weight values to minimise quantisation error; (ii) double quantisation, which reduces memory overhead by quantising the quantisation constants themselves; and (iii) paged optimisers, which manage GPU memory spikes during gradient computation by offloading optimiser states to CPU memory as needed. Together, these techniques allow a 3-billion-parameter model to be fine-tuned on a single consumer-grade or free-tier cloud GPU (e.g., an NVIDIA T4 with 16 GB VRAM), which is the hardware configuration used in the present study.
QLoRA has been widely adopted in the research community as a practical baseline for resource-efficient LLM adaptation, and its effectiveness has been validated across tasks including instruction tuning, summarisation, and code generation [8]. Its application to domain-specific numerical prediction tasks such as that undertaken in the present study represents a natural extension of this prior work.

2.4.4 Supervised Fine-Tuning (SFT)

The fine-tuning paradigm employed in the present study is Supervised Fine-Tuning (SFT), in which the model is trained on a dataset of input–output pairs to maximise the likelihood of the correct output given the input under the standard causal language modelling objective. This approach is distinguished from reinforcement learning from human feedback (RLHF) [23] and direct preference optimisation (DPO) [28], which require preference-

labelled data or reward models. SFT on labelled (prompt, completion) pairs is straightforward to implement using frameworks such as the Hugging Face TRL library [33], which provides the `SFTTrainer` class used in the present study.

2.4.5 Checkpoint Selection and Early Stopping

A practical concern in any supervised fine-tuning procedure is determining which saved model state, or *checkpoint*, to retain for evaluation and deployment. When training proceeds for multiple epochs or a large number of steps, the model's training loss typically decreases monotonically, while the validation loss may reach a minimum at an intermediate point before increasing again as the model begins to overfit the training distribution. This phenomenon, well-documented in classical neural network training [26], is equally relevant to the fine-tuning of large pre-trained models on small domain-specific corpora. Hu et al. (2022) and subsequent work on LoRA-based fine-tuning have noted that PEFT methods are comparatively resistant to overfitting due to their small number of trainable parameters, but are not immune to it [14]. In practice, the checkpoint that minimises validation loss does not always coincide with the final checkpoint, particularly when training data is limited or when the domain distribution is highly specific. Selecting the minimum validation-loss checkpoint — commonly referred to as the *best checkpoint* — is therefore a standard component of rigorous fine-tuning practice [20].

2.5 LLMs in Sports Analytics and Forecasting

Having reviewed the technical foundations of parameter-efficient LLM fine-tuning, the following section surveys the application domain — sports analytics and numerical forecasting — and positions the present study within this emerging literature.

2.5.1 Sports Analytics with Machine Learning

The application of machine learning to sports prediction has a rich and growing literature across football, basketball, baseball, and tennis [12]. In cricket specifically, researchers

have applied classification models to predict match outcomes [2], regression models to predict individual player performance [17], and player evaluation metrics designed specifically for the T20 format [7]. These works collectively establish that historical match data, when appropriately processed, contains sufficient signal for useful predictive models.

2.5.2 Language Models for Structured and Tabular Data

A growing body of work investigates the use of LLMs as predictors for structured, tabular, or time-series data by serialising numerical records into natural-language representations. Hegselmann et al. (2023) showed that a fine-tuned GPT-style model could outperform gradient-boosted decision trees on tabular classification tasks when data were serialised as natural-language sentences, particularly in low-data regimes [13]. Gruver et al. (2023) demonstrated that LLMs can serve as zero-shot time-series forecasters by encoding numerical sequences as token strings, achieving performance competitive with specialised forecasting models on standard benchmark datasets [11].

These results demonstrate that LLMs can perform effectively on structured numerical prediction tasks when domain-specific data are serialised into natural-language representations, achieving competitive performance without task-specific architectural modifications. Whether this capability transfers to the highly domain-specific context of in-game T20 score prediction is a central question of the present research.

2.5.3 LLMs Applied to Sports-Specific Tasks

Direct applications of LLMs to sports prediction tasks remain limited in the published literature, though interest in this area is growing. Huckerby et al. (2024) explored GPT-based models for football match outcome prediction using natural-language summaries of team statistics, finding modest but consistent improvements over baseline classifiers when the model was fine-tuned on domain-specific data [15]. However, to the best of the author's knowledge, no prior work has explicitly investigated the use of instruction-tuned or fine-tuned LLMs for in-game T20 score prediction using natural-language prompt

representations.

2.6 Research Gap and Positioning of the Present Study

The literature reviewed in the preceding sections reveals a clear trajectory: from hand-crafted statistical models, through classical machine learning with numeric feature vectors, to the emerging use of pre-trained language models for structured prediction tasks. Within this trajectory, several specific gaps are identifiable.

First, existing cricket score prediction models treat match state as a numeric vector and do not exploit the semantic expressiveness of natural language as an input modality. Second, while LLMs have been applied to tabular data prediction and time-series forecasting in general domains, their application to in-game cricket score prediction has not been systematically investigated. Third, the comparative evaluation of a fine-tuned small open-source LLM against a large proprietary frontier model on a sports forecasting task has not, to the author's knowledge, been undertaken in the cricket analytics literature. Fourth, the importance of checkpoint selection in LLM fine-tuning for domain-specific prediction tasks has not been empirically demonstrated in the sports analytics literature.

The present study addresses these gaps by: (i) constructing a natural-language prompt schema that encodes rich in-game context; (ii) fine-tuning a compact, open-source LLM (Llama 3.2-3B) using QLoRA on a domain-specific dataset of historical T20 innings snapshots; (iii) systematically comparing the minimum validation-loss checkpoint against the final training checkpoint to identify the optimal deployment model; and (iv) benchmarking the fine-tuned model against both a zero-shot baseline and a frontier model (GPT-4.1-nano) on a held-out temporal test set. In doing so, this research contributes to both the cricket analytics literature and the growing body of work on domain-specific LLM adaptation for quantitative prediction tasks.

Chapter 3

Materials and Methods

3.1 Introduction

This chapter describes the complete pipeline employed in the present study, from raw data acquisition through to the evaluation framework used to assess model performance. The pipeline comprises five stages: (i) data collection and preprocessing; (ii) feature engineering; (iii) prompt schema design and dataset construction; (iv) model architecture and fine-tuning; and (v) evaluation. Each stage is described in sufficient detail to support reproducibility.

3.2 Data Collection and Preprocessing

3.2.1 Source Dataset

Match data were obtained from Cricsheet [6], a publicly accessible repository that distributes structured ball-by-ball records of international cricket matches in JSON format. The full dataset of international men's T20 matches was downloaded as a ZIP archive from `https://cricsheet.org/downloads/t20s_json.zip`. After extraction, the male match directory contained **3,113 JSON files**, covering matches from approximately 2005 through January 2026.

Each JSON file represents a single match and adheres to Cricsheet schema versions 1.0.0 or 1.1.0. The top-level structure contains three keys: `meta` (schema version and file creation date), `info` (match-level metadata including teams, venue, date, toss, and outcome), and `innings` (an ordered list of innings objects, each comprising an ordered list of overs,

and each over containing an ordered list of deliveries with batter, bowler, runs, extras, and wicket information). The JSON files were parsed into typed Python objects using a set of Pydantic v2 data models defined in `forecaster/models.py`, providing automatic schema validation and convenient attribute access.

3.2.2 Filtering Criteria

Two sequential filtering steps were applied before any features were extracted.

Step 1 — Exclusion of no-result matches. Matches where the `info.outcome.result` field equals `"no result"` were excluded. Such matches are those abandoned due to weather or other circumstances before a result could be determined, and they do not produce a complete or meaningful first-innings score.

Step 2 — Exclusion of incomplete first innings. Matches where the first innings had fewer than 20 overs bowled *and* fewer than 10 wickets fallen were excluded. This filter removes innings that were reduced by Duckworth–Lewis–Stern adjustments or prematurely terminated by agreement, retaining only innings that represent a natural T20 conclusion (either all twenty overs bowled or the batting side dismissed). After both filters, approximately **2,895 matches** were retained, yielding **28,954 labelled examples** across all data splits (ten per match).

3.2.3 Venue Name Normalisation

The Cricsheet JSON files record venue names as free-text strings that exhibit substantial inconsistency across matches: the same physical ground may appear under multiple spellings, with or without a city or country suffix, or with minor punctuation differences. Left unresolved, these variants would be treated as distinct venues, fragmenting the historical match history used to compute per-venue par scores and reducing the reliability of the venue statistics.

A conservative venue name normalisation step was applied after filtering, using a two-rule algorithm implemented in `research/02_data_processing_vanues.ipynb`. Venues

were processed in ascending order of name length so that the shorter, unqualified form of a name was established as canonical before any longer variant was encountered.

Rule 1 — Exact prefix with location suffix. If venue name A is an exact string prefix of venue name B, and the characters immediately following A in B begin with a comma, B is mapped to A. This rule handles the dominant inconsistency pattern in the dataset, in which the same ground appears both without and with a city or country qualifier:

"Eden Gardens" $\leftarrow$ *"Eden Gardens, Kolkata"*

"Wankhede Stadium" $\leftarrow$ *"Wankhede Stadium, Mumbai"*

"SuperSport Park" $\leftarrow$ *"SuperSport Park, Centurion"*

Rule 2 — High-threshold fuzzy match. If two venue names have a token-sorted fuzzy similarity ratio of 95 or above (computed using the `fuzzywuzzy` library), the shorter name is adopted as canonical. The threshold was set deliberately high to avoid incorrect merges between genuinely different grounds; in practice this rule resolves only minor punctuation differences such as a full stop in place of a space:

"M Chinnaswamy Stadium" $\leftarrow$ *"M.Chinnaswamy Stadium"*

The resulting mapping was stored in a `venue_mapping` table in the SQLite database. The raw `first_innings_sections` table was joined against this mapping to produce a cleaned `first_innings_sections_clean` table. Venue normalisation reduced the number of distinct venue strings from **322** to **236**, collapsing **86** variant names into their canonical equivalents. All subsequent feature engineering and par score computation was performed on the cleaned venue labels.

3.3 Feature Engineering

3.3.1 Innings Segmentation

Each qualifying first innings was partitioned into ten non-overlapping two-over sections. Section k ($k = 1, 2, \ldots, 10$) corresponds to the completion of overs $2k - 2$ and $2k - 1$

(zero-indexed). A feature snapshot is captured at the end of each section, so a single match contributes ten rows to the dataset, each representing a distinct in-game state from which the final innings total is to be predicted.

3.3.2 Per-Section Features

Sixteen features were extracted at the end of each two-over section, as summarised in Table 3.1. Cumulative features are computed over the entire innings up to the end of the section; section-level features are computed over the two overs of the section only.

Table 3.1: Features extracted per two-over section

Feature	Definition
`section`	Section index (1–10)
`overs_completed`	Total integer overs elapsed
`balls_remaining`	$(20 - \texttt{overs\_completed}) \times 6$
`cumulative_runs`	Total runs scored up to end of section
`cumulative_wickets`	Total wickets lost up to end of section
`cumulative_run_rate`	$\texttt{cumulative\_runs}/\texttt{overs\_completed}$
`runs_in_section`	Runs scored in this two-over section
`dot_balls_so_far`	Cumulative deliveries on which no runs were scored
`dot_balls_last_2_overs`	Dot balls in this section
`fours_hit`	Cumulative count of batting fours
`sixes_hit`	Cumulative count of batting sixes
`boundaries_last_2_overs`	Fours plus sixes in this section
`powerplay_completed`	Boolean: True if $\texttt{overs\_completed} > 6$
`avg_score`	Venue historical average first-innings total
`par_score_filled`	Venue par score (see Section 3.3.3)
`remaining_batting_strength_score`	Weighted sum of remaining players (see Section 3.3.6)

Note: `avg_score` is an intermediate quantity used to derive `par_score_filled` and is not included in the natural-language prompt. `boundaries_last_2_overs` is extracted and stored for reference; boundary information is represented in the prompt by the individual `fours_hit` and `sixes_hit` fields.

3.3.3 Venue Par Score

A per-venue par score was computed as a proxy for the typical winning total at each ground. For venues with ten or more historical matches in the dataset, the par score was defined as the 52nd percentile of first-innings totals across all qualifying matches at that venue:

$$\text{par\_score}_v = Q_{0.52}\big(\{s_i : \text{venue}(i) = v\}\big) \tag{3.1}$$

where s_i denotes the first-innings total of match i. The 52nd percentile was selected as it corresponds approximately to the score at which a batting-first team achieves an even chance of winning, consistent with the resource-allocation interpretation underlying the DLS method [10]. For venues with fewer than ten historical matches in the dataset (100 out of 236 unique venues after normalisation), the par score was set equal to the rounded historical average first-innings total at that venue as a fallback. The resulting `par_score_filled` column was stored in a SQLite database and joined to the section-level feature table.

3.3.4 Player Batting and Bowling Statistics

Per-player career statistics were aggregated across all qualifying first innings in the dataset prior to classification. The following statistics were recorded for each player:

Batting: total runs, balls faced (excluding wide deliveries), fours, sixes, dismissal count, dot balls faced (deliveries on which no runs were scored, excluding wides), batting average ($\bar{r}_{\text{bat}} = \text{total runs}/\text{dismissals}$), and batting strike rate ($\text{SR}_{\text{bat}} = 100 \times \text{total runs}/\text{balls faced}$).

Bowling: legal deliveries bowled (excluding wides and no-balls for ball count, but including them for run count), runs conceded (including all extras), wickets taken (run-outs excluded), wides, no-balls, dot balls bowled, bowling average ($\bar{r}_{\text{bowl}} = \text{runs conceded}/\text{wickets}$), and economy rate ($\text{econ} = \text{runs conceded}/\text{overs bowled}$).

All statistics were stored in a SQLite database table (`first_innings_player_stats`) and accessed by the player classification module.

3.3.5 Player Classification

Each player in the dataset was assigned one of five role classifications based on their career-aggregated statistics, using the function `classify_t20i_player` de-

fined in `forecaster/player_classification.py`. A player was assigned the label `INSUFFICIENT_DATA` if they had appeared in fewer than 15 matches, or if their total runs were below 250 and their total wickets were below 10. For all other players, the following rules were applied in priority order (first matching rule determines the classification):

1. **BOWLER:** overs per match ≥ 3, economy ≤ 8.2, bowling average ≤ 26, batting average < 20.
2. **ALL_ROUNDER (bowling-dominant):** overs per match ≥ 2, total wickets ≥ 25, batting average ≥ 20, batting strike rate ≥ 120, economy ≤ 8.8.
3. **ALL_ROUNDER (batting-dominant):** overs per match ≥ 1, total wickets ≥ 20, batting average ≥ 25, batting strike rate ≥ 130.
4. **BATTER:** batting average ≥ 28, batting strike rate ≥ 125, overs per match < 0.5.
5. **UTILITY_PLAYER:** all remaining players who meet the minimum data threshold.

3.3.6 Remaining Batting Strength Score

At the end of each two-over section, the set of players from the batting team's eleven-player lineup who had not yet been dismissed was identified from the delivery records. The *Remaining Batting Strength Score* (RBSS) was defined as a weighted sum over this set:

$$\text{RBSS} = \sum_{p \in \mathcal{R}} w\big(\text{class}(p)\big) \tag{3.2}$$

where $\mathcal{R}$ is the set of undismissed players at the end of the section, $\text{class}(p)$ is the player's assigned classification, and the weight function w is defined in Table 3.2. The score was rounded to two decimal places.

The weights were designed to be stable, simple, and rounded, avoiding arbitrary decimal values that would be difficult for a language model to interpret consistently. `BATTER` is set to 1.00 as the baseline unit of batting value. `ALL_ROUNDER` receives 0.80, reflecting

Table 3.2: Player classification weights used in the Remaining Batting Strength Score

Classification	Weight	Rationale
BATTER	1.00	Baseline reference unit; primary run-scoring resource in T20 cricket
ALL_ROUNDER	0.80	Strong middle-order contributors with high strike rates, but less consistent than specialist batters
UTILITY_PLAYER	0.65	Mixed skill set and inconsistent output; positioned between batter and tail
INSUFFICIENT_DATA	0.45	Unknown capability; conservative mid-point to avoid over- or under-estimating unproven players
BOWLER	0.30	Tail-end batters with limited scoring expectation; retains a non-zero weight to capture occasional late hitting

that T20 all-rounders frequently bat in positions 5–7 with high strike rates but lower consistency than specialist top-order batters. `UTILITY_PLAYER` is assigned 0.65, occupying the mid-point between reliable batting and tail-end contributions. `BOWLER` is assigned 0.30 rather than zero, as specialist bowlers in T20 cricket can contribute cameo innings in the death overs. The `INSUFFICIENT_DATA` weight of 0.45 is set at a conservative mid-point: assigning a low weight would systematically underestimate young or debut players who may be capable batters, while assigning a high weight would be overly optimistic about genuinely inexperienced or peripheral players.

To illustrate the computation, consider a match state at the end of over 12 in which the following seven players remain undismissed: one BATTER, two ALLROUNDERs, one UTILITY_PLAYER, and three BOWLERs. The RBSS at this point is:

$$\text{RBSS} = (1\times1.00)+(2\times0.80)+(1\times0.65)+(3\times0.30) = 1.00+1.60+0.65+0.90 = 4.15 \tag{3.3}$$

This score is encoded directly in the natural-language prompt as `Remaining Batting Strength Score: 4.15`, providing the model with a single numerical signal that integrates batting depth, batting quality, and order position without exposing a noisy list of

individual player names. The RBSS therefore encodes not merely the number of wickets remaining but the batting quality of the players yet to bat, providing a quantitative estimate of the residual scoring capacity of the innings.

3.4 Prompt Schema and Dataset Construction

3.4.1 Natural-Language Prompt Design

Each in-game state snapshot was serialised into a structured natural-language prompt. The prompt template is shown below, with placeholders in braces:

```
Match type: T20
Venue: {venue}
Venue par score: {par_score_filled}
Batting team: {team}

Overs completed: {overs_completed}
Balls remaining: {balls_remaining}
Runs scored: {cumulative_runs}
Wickets lost: {cumulative_wickets}
Current run rate: {cumulative_run_rate}

Runs in last 2 overs: {runs_in_section}
Dot balls so far: {dot_balls_so_far}
Dot balls in last 2 overs: {dot_balls_last_2_overs}

Fours hit: {fours_hit}
Sixes hit: {sixes_hit}

Powerplay completed: {Yes|No}
```

`Remaining Batting Strength Score: {RBSS}`

`Final 1st innings score:`

The completion (target output) is the actual final first-innings total as a plain integer string (e.g., `"173"`). This design frames the prediction task as a text completion problem: given the in-game state description, the model must continue the prompt with a plausible score. Prompts were tokenised using the `meta-llama/Llama-3.2-3B` tokenizer; the maximum observed prompt length was 148 tokens, and the maximum sequence length used during training was set to 256 tokens to provide sufficient headroom for the prompt and completion combined.

A second prompt variant, used exclusively for frontier model evaluation, omits the final line `"Final 1st innings score:"`, instead presenting the match state as a self-contained paragraph (the `summary` field). The frontier model receives an explicit system instruction to respond with a number only, and its response is therefore not a continuation but a standalone prediction.

3.4.2 Dataset Split

The dataset was partitioned chronologically by match date to prevent any form of data leakage and to simulate realistic deployment conditions, in which the model is evaluated on matches it could not have encountered during training. The partitioning boundaries and approximate sizes are given in Table 3.3.

Table 3.3: Temporal train/validation/test split

Split	**Date range**	**Approx. matches**	**Approx. rows**
Train	Up to 2024-12-31	2,408	24,080
Validation	2025-01-01 – 2025-06-30	237	2,370
Test	2025-07-01 onwards	250	2,500
Total		2,895	28,950

The resulting datasets were published to the Hugging Face Hub under the identifiers

`ratnayakatilanka/t20_score_prediction_prompts` (prompt/completion pairs for fine-tuning and fine-tuned model evaluation) and `ratnayakatilanka/t20_score_prediction_summary` (summary/score pairs for frontier model evaluation).

3.5 Model Architecture and Fine-Tuning

3.5.1 Base Model

The base model selected for this study is *Llama 3.2-3B* (`meta-llama/Llama-3.2-3B`), a 3-billion-parameter decoder-only transformer released by Meta AI [21]. This model was chosen for three reasons: (i) its parameter count is sufficiently small to permit fine-tuning on consumer-grade hardware using quantisation; (ii) it is publicly available under a permissive research licence; and (iii) its scale is representative of the class of resource-efficient models that would be considered for practical deployment in resource-constrained settings.

To reduce GPU memory requirements, the model was loaded with 4-bit quantisation using the `BitsAndBytesConfig` configuration. Specifically, NormalFloat 4-bit (NF4) quantisation was applied to all frozen base model weights, together with double quantisation (quantising the quantisation constants themselves to further reduce memory overhead) [8]. The compute dtype was set to `float16` on NVIDIA T4 hardware.

3.5.2 LoRA Adapter Configuration

Low-Rank Adaptation (LoRA) adapter layers [14] were injected into the four attention projection matrices of every transformer layer: `q_proj`, `v_proj`, `k_proj`, and `o_proj`. The adapter configuration used for the training run reported in this study (LITE mode, Kaggle T4 GPU) is given in Table 3.4.

Table 3.4: LoRA adapter configuration (LITE mode)

Parameter	Value
Rank (r)	16
Alpha (α)	32
Dropout	0.1
Bias	none
Task type	CAUSAL_LM
Target modules	`q_proj, v_proj, k_proj, o_proj`

3.5.3 Training Procedure

Fine-tuning was conducted using the Supervised Fine-Tuning (SFT) paradigm with the `SFTTrainer` class from the Hugging Face TRL library [33]. The training objective is the standard causal language modelling loss computed over the completion tokens only. Training was performed on the **Kaggle** free-tier platform using an NVIDIA T4 GPU (16 GB VRAM). The key hyperparameters are listed in Table 3.5. Training progress was logged to **Weights & Biases** under the project name `t20_score_prediction`. The resulting LoRA adapter weights were saved to the private Hugging Face repository `ratnayakatilanka/t20_score_prediction-_keggle-2026-04-18_04.16.40-lite`.

Table 3.5: Training hyperparameters (LITE mode, Kaggle T4)

Hyperparameter	Value
Epochs	1
Per-device batch size	4
Gradient accumulation steps	2
Effective batch size	8
Learning rate	2×10^{-4}
LR scheduler	cosine
Warmup ratio	0.03
Weight decay	0.001
Max gradient norm	0.3
Max sequence length (tokens)	256
Optimizer	`paged_adamw_8bit`
Evaluation interval	every 100 steps
Checkpoint save limit	10
Precision	fp16

3.5.4 Checkpoint Selection

The training curves recorded by Weights & Biases for the Kaggle fine-tuning run are shown in Figures 3.1–3.3. Figure 3.1 shows the training loss (`train/loss`), which decreases steadily across the training steps. Figure 3.2 shows the validation loss (`eval/loss`), which reaches its minimum of **1.856** at step 800 before showing a marginal upturn, indicating the onset of overfitting. Figure 3.3 shows the cumulative evaluation runtime (`eval/runtime`), confirming that validation was conducted at regular 100-step intervals throughout training.

Figure 3.1: Training loss (`train/loss`) over training steps recorded by Weights & Biases for the Kaggle fine-tuning run (`t20_score_prediction_keggle`, run ID: `v64wpa10`).

Validation loss was monitored at every 100 training steps using a held-out set comprising the chronologically first 50 rows of the validation split. This fixed subsample was chosen to minimise per-evaluation GPU time on the Kaggle T4 platform; the resulting loss estimate is noisier than a full-split evaluation, and the potential effect on checkpoint selection reliability is discussed in Chapter 5. The checkpoint at **step 800** achieved the minimum observed validation loss of **1.856** and was identified as the best checkpoint. Its Hugging Face Hub revision SHA is `00be3f3f2cb4af3cd727c68284ca3dcbad7656a4`. The fine-tuned model was evaluated at both this checkpoint and at the final checkpoint (end of epoch) to determine which should be used for the primary evaluation. The comparison is

Figure 3.2: Validation loss (`eval/loss`) over training steps. The minimum validation loss of 1.856 is achieved at step 800, which is selected as the best checkpoint.

reported in Chapter 5.

3.6 Evaluation Framework

3.6.1 Experimental Conditions

Three experimental conditions were evaluated on the held-out test set:

1. **Zero-shot LLaMA 3.2-3B (baseline):** The unmodified base model was prompted with the full in-game-state prompt and asked to generate a continuation. No domain-specific training was applied.

2. **Fine-tuned LLaMA 3.2-3B:** The QLoRA-adapted model was loaded with the step-800 checkpoint adapter and evaluated on the same test set under identical inference conditions.

3. **Zero-shot GPT-4.1-nano (frontier reference):** The `openai/gpt-4.1-nano` model was queried via the LiteLLM Python library [4] using the summary prompt variant. The model was prompted with a user-turn instruction prepended to the match-state context: *"Predict the final T20 first-innings score in runs. Respond with the number only, no explanation."*

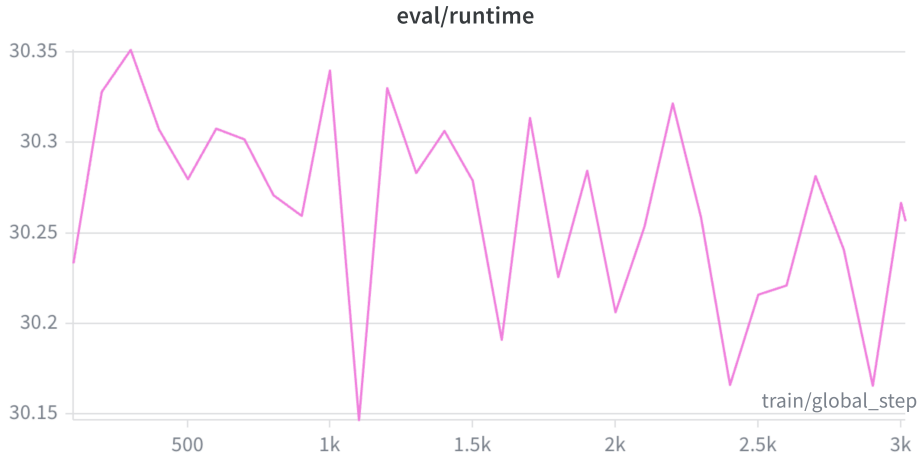

Figure 3.3: Cumulative evaluation runtime (`eval/runtime`) over training steps, confirming that validation was conducted at regular 100-step intervals throughout training.

3.6.2 Inference Configuration

For the fine-tuned LLaMA conditions, inference was performed using greedy decoding (`do_sample=False`) with a maximum of 8 new tokens generated beyond the prompt; in practice the fine-tuned model consistently generated 2–3 tokens. For the zero-shot LLaMA baseline, 15 new tokens were permitted to allow the untuned model greater latitude to produce a recognisable integer. Only the tokens generated after the end of the input prompt were decoded; the prompt tokens themselves were excluded from the output. For GPT-4.1-nano, the API was called with default sampling settings, and the raw text response was processed identically to the LLaMA output.

3.6.3 Output Post-processing

Raw model outputs were processed with a two-step regex extraction procedure to isolate the predicted score:

1. **Primary pattern:** Extract the first standalone 2-or-3-digit number matching `\b\d{2,3}\b`. This targets the typical range of T20 first-innings totals (roughly 60–250 runs).

2. **Fallback pattern:** If no match is found, extract any numeric value (integer or decimal) using `[-+]?\d*\.\d+|\d+`.

3. **Default:** If neither pattern yields a result, the predicted score is set to 0.

3.6.4 Performance Metrics

Model performance was assessed using three standard regression metrics computed over the test set:

- **Mean Absolute Error (MAE):** $\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|\hat{y}_i - y_i|$, reported in runs. This is the primary metric, reflecting the average prediction error in the natural unit of the task.

- **Mean Squared Error (MSE):** $\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$, penalising large individual errors more heavily than MAE.

- **Coefficient of Determination (R^2):** $R^2 = 1 - \frac{\sum(\hat{y}_i - y_i)^2}{\sum(y_i - \bar{y})^2}$, reported as a percentage. A value of 100% indicates perfect prediction; a value of 0% indicates a model no better than predicting the mean; negative values indicate a model worse than the mean predictor.

All metrics were computed using the `scikit-learn` library [24]. Evaluations were conducted on 200 datapoints (the default LITE-mode setting) and on 500 datapoints for the fine-tuned model checkpoint comparison. The larger evaluation set provides a more reliable estimate of generalisation performance by reducing the influence of the chronological ordering of the test set.

3.7 Serverless Inference Deployment

To validate end-to-end deployment, a three-tier inference stack was implemented: a serverless GPU backend (Modal [22]), a Gradio research prototype deployed to Hugging Face Spaces [16], and a production-quality React dashboard deployed to Azure Static Web

Apps. The architecture separates inference compute (GPU, Modal) from presentation (CPU-only, static hosting) so that GPU cost scales proportionally with request volume. The subsections below describe each tier in order of implementation complexity. The deployment validated that the LoRA adapter artefact published to Hugging Face can be loaded on a Modal-hosted container, invoked through a callable remote function, and exposed via publicly accessible interfaces suitable for both research and broadcast contexts, in a fully reproducible cloud-hosted environment.

3.7.1 Ephemeral Inference Function

The first deployment pattern used Modal's `@app.function` decorator to create a stateless ephemeral function (`t20_scorer_ephemeral.py`) that loads the base model, applies the LoRA adapters, generates a prediction, and then releases GPU resources. The container image used `debian_slim` as a base with `torch`, `transformers`, `bitsandbytes`, `accelerate`, and `peft` installed at build time. The base model (`meta-llama/Llama-3.2-3B`), fine-tuned adapter repository, and specific checkpoint revision SHA (`00be3f3f`) were hard-coded to ensure reproducibility. An NVIDIA T4 GPU was requested for each invocation. On first invocation, model loading required approximately 30 seconds (254 quantised weight shards); generation itself completed in under one second thereafter.

3.7.2 Persistent Warm Service

The second deployment pattern used Modal's `@app.cls` decorator, together with a `@modal.enter()` setup method (`t20_scorer_service.py`), which loads the base model and LoRA adapters exactly once when the container starts. Subsequent calls to the `predict` method reuse the in-memory model, reducing per-request latency to approximately 2 seconds. Model weights were cached in a Modal persistent Volume (`hf-hub-cache`) to eliminate repeated downloads from Hugging Face on container cold starts. The `MIN_CONTAINERS` parameter controls scaling behaviour: a value of `0` permits

scale-to-zero after two minutes of inactivity (cold start ≈30 s); a value of `1` maintains a permanently warm container (warm response ≈2 s).

The service was deployed to Modal's platform and invoked remotely using the `predict.remote()` method to confirm end-to-end functionality. For a sample mid-innings prompt (Melbourne Cricket Ground, Australia batting, 10 overs completed, score 82/2, current run rate 8.2), both the ephemeral function and the persistent service produced a predicted final score of 165 runs, consistent with the benchmark evaluation results.

3.7.3 Gradio Web Interface and Hugging Face Spaces Deployment

A web-based interactive front-end was developed using the Gradio library [1] and deployed to Hugging Face Spaces [16] as the user-facing tier of the deployment stack. The application (`app.py`) accepts the raw match state as structured form inputs — venue, par score, batting team, overs completed, current score, wicket count, boundary counts, recent-over statistics, and the breakdown of remaining batting roles — and computes the current run rate and Remaining Batting Strength Score automatically using the same weight table as the training pipeline (Section 3.3.6) before constructing the prompt. The completed prompt is then dispatched to the deployed Modal service via `scorer.predict.remote()`, and the predicted final score is displayed to the user. An optional JSON upload facility allows a fully specified match-state object to populate all form fields at once, enabling batch inspection of individual test-set rows.

The Space was provisioned programmatically by `deploy_space.py`, which uses the Hugging Face Hub API to create a Gradio-type Space under the account `ratnayakatilanka`, upload `app.py` and `requirements.txt`, and print the direct link to the Secrets settings page where the Modal authentication tokens must be set. This script ensures that the Space can be re-deployed from a clean state without manual browser interaction beyond the one-time token configuration step.

The three-tier architecture connects the Hugging Face Space, which hosts the Gradio front-end and handles HTTP traffic, with the Modal warm service, which is called over the net-

work to generate predictions, and the Hugging Face model repository, from which model weights are fetched on container cold start and cached in a Modal persistent Volume for subsequent requests. This separation of concerns means that inference GPU cost is borne only by Modal and is proportional to actual usage, while the Space itself runs without a GPU and incurs no compute charge at idle.

3.7.4 Production React Web Application

A production-quality web interface, the *T20 Intelligence Console* (`webapp/`), was developed to supersede the Gradio prototype as the user-facing tier of the deployment stack. The application was built with React 18, Vite 5, and Tailwind CSS 3, and presents a dark-mode broadcast-style dashboard composed of modular components. `HeroPrediction` displays the predicted score with a circular gauge (`ScoreGauge`) alongside live run-rate and wicket statistics; `ConfidenceMeter` renders a visual confidence strip when the backend returns a probability estimate; `MomentumChart` plots the cumulative run trajectory; `LineupSelector` accepts the per-role batting-lineup breakdown for RBSS computation; and `VenueInsights` surfaces venue par score context. A `JsonUploadZone` component allows a fully specified match-state JSON object to populate all form fields at once.

All cricket domain logic — the RBSS weight table, run-rate calculation, and prompt builder — is re-implemented in `src/lib/cricket.js`, mirroring the Python weight table and run-rate calculation from `research/app.py`. The prompt format in `cricket.js` includes an additional batting lineup breakdown section and uses shortened field labels (`Fours:`, `Sixes:`, `Batting strength:`) that differ from the training prompt format; the React interface therefore functions as a demonstration prototype rather than a production-ready inference client that reproduces the exact training distribution. The thin HTTP client in `src/api.js` calls `POST /api/predict` and expects a JSON response containing a mandatory `score` field alongside optional `confidence` and `range` fields; the UI degrades gracefully when the optional fields are absent. When no `VITE_PREDICT_URL` environment variable is configured, the client falls back to a local heuristic mock that projects the final

score from run rate, recent momentum, wickets fallen, and batting strength, enabling the full dashboard to be demoed without a live inference backend.

The production backend is implemented as a Modal `@asgi_app()` ASGI function (`predict_api` in `t20_scorer_service.py`) that wraps the `Scorer.predict` method behind a FastAPI `POST /predict` endpoint with CORS headers, enabling direct HTTP invocation from the React client. The application is built and hosted via a GitHub Actions workflow (`.github/workflows/deploy.yml`) that is triggered automatically on every push to the `main` branch that touches `webapp/` or `infra/`. The workflow authenticates to Azure using OpenID Connect federated identity credentials, eliminating the need for long-lived stored secrets. It first deploys an Azure Bicep template (`infra/main.bicep`) that provisions a Free-tier Azure Static Web Apps resource named `t20-intelligence-console` in the East Asia region. It then retrieves the Static Web Apps deployment token via the Azure CLI, builds the frontend with `npm run build`, and publishes the `dist/` output using the `Azure/static-web-apps-deploy` action. A `staticwebapp.config.json` file placed in `webapp/public/` instructs the platform to rewrite any unresolved URL path to `/index.html`, enabling client-side React Router navigation without 404 errors on deep links. The Vite `base` path is set to `/` (root), consistent with Azure Static Web Apps serving the application from the domain root rather than a subdirectory. The application is publicly accessible at `https://mango-ocean-00260de00.7.azurestaticapps.net/`. Because the deployed artefact consists entirely of static HTML, CSS, and JavaScript files, the platform incurs no compute charge at idle; the only variable cost in the complete stack remains the Modal GPU compute time consumed per inference request.

Chapter 4

Results

4.1 Introduction

This chapter presents the empirical results of all experimental conditions evaluated in the present study. Section 4.2 reports the performance of the zero-shot *Llama 3.2-3B* base model as the primary baseline. Section 4.3 presents the results of the fine-tuned model under a comparison of checkpoint selection strategies. Section 4.4 reports the performance of the frontier model *GPT-4.1-nano*. Section 4.5 provides a consolidated cross-condition comparison. All results are reported on the held-out temporal test set using Mean Absolute Error (MAE, in runs), Mean Squared Error (MSE), and the coefficient of determination (R^2, as a percentage).

4.2 Zero-Shot Baseline: Llama 3.2-3B

The unmodified *Llama 3.2-3B* base model was evaluated in a zero-shot setting — without any domain-specific fine-tuning — to establish a pre-adaptation baseline. In this condition, the model received the full natural-language match-state prompt and was required to generate a completion representing the predicted first-innings total.

The zero-shot baseline experiments were conducted on Google Colab (NVIDIA T4 GPU) using 4-bit NF4 quantisation and greedy decoding. The model was evaluated on 200 test datapoints drawn from the temporal test split. Outputs were post-processed using the two-step regex extraction procedure described in Section 3.6.3 to extract a numeric prediction from each generated continuation.

The results are summarised in Table 4.1.

Table 4.1: Zero-shot *Llama 3.2-3B* baseline evaluation results (200 datapoints)

Model	**MAE (runs)**	**MSE**	R^2 **(%)**
Llama 3.2-3B (zero-shot)	71.31	7,463	-317.4

The untuned model exhibits extremely poor predictive performance, with an MAE of **71.31 runs**, an MSE of **7,463**, and an R^2 of $-$**317.4%**. The sharply negative R^2 indicates that the model's predictions are substantially worse than simply predicting the mean first-innings score for every input — a result consistent with a model that has not been trained to associate structured T20 match-state descriptions with plausible final totals. Many outputs from the zero-shot model were incoherent, out of range, or consisted of repeated tokens rather than a recognisable integer score, which the post-processing regex resolved to 0 (the default fallback), inflating the error further. These results confirm that the pre-trained model carries essentially no usable domain knowledge for this task in the zero-shot setting, and they establish a clear lower bound for comparison with the fine-tuned and frontier conditions.

4.3 Fine-Tuned Llama 3.2-3B: Checkpoint Comparison

4.3.1 Overview

The fine-tuned model was evaluated under four sub-conditions arising from the combination of two checkpoint variants (step-800 minimum-loss checkpoint vs. final epoch checkpoint) and two test-set sizes (200 and 500 datapoints). This design isolates two sources of variation: (i) the effect of checkpoint selection on generalisation, and (ii) the reliability of the evaluation estimate as a function of sample size. The results across all four sub-conditions are summarised in Table 4.2.

4.3.2 Effect of Checkpoint Selection at 200 Datapoints

At 200 test datapoints, the final epoch checkpoint appears to outperform the step-800 checkpoint across all three metrics: MAE of 16.12 runs versus 17.05 runs, MSE of 618

Table 4.2: Fine-tuned *Llama 3.2-3B* checkpoint comparison on the temporal test set

Checkpoint	**N**	**MAE (runs)**	**MSE**	R^2 **(%)**
Step-800 (min. eval/loss = 1.856)	200	17.05	601	66.4
Final epoch checkpoint	200	**16.12**	618	65.4
Step-800 (min. eval/loss = 1.856)	500	**16.51**	**543**	**68.3**
Final epoch checkpoint	500	16.54	546	68.1

versus 601, and R^2 of 65.4% versus 66.4%. However, this apparent advantage is contradicted by the MSE comparison, which favours the step-800 checkpoint (601 vs. 618) despite the latter having the higher MAE. This inconsistency is indicative of noise in the evaluation estimate. The test set is ordered chronologically, and 200 datapoints represent a narrow, temporally contiguous window of matches, making the evaluation sensitive to the particular scoring patterns of the matches included in that window.

4.3.3 Effect of Checkpoint Selection at 500 Datapoints

At 500 test datapoints, the ordering of results reverses: the step-800 checkpoint achieves lower MAE (16.51 vs. 16.54 runs), lower MSE (543 vs. 546), and higher R^2 (68.3% vs. 68.1%). Although the absolute differences are small, the consistency across all three metrics at the larger sample size provides stronger evidence that the step-800 checkpoint generalises marginally better. The improvement in all metrics when moving from 200 to 500 datapoints (for both checkpoints) further confirms that the 200-point evaluation was subject to sample noise.

Based on this analysis, the **step-800 checkpoint** (revision SHA `00be3f3f`) is adopted as the primary fine-tuned model for the cross-condition comparison in Section 4.5, evaluated on 500 datapoints. This choice is consistent with the standard practice of selecting the minimum validation-loss checkpoint for deployment [26].

4.3.4 Prediction Accuracy at 500 Datapoints

The fine-tuned model (step-800, 500 datapoints) achieves an average prediction error of **16.51 runs**, an MSE of **543**, and an R^2 of **68.3%**. An R^2 of 68.3% indicates that the model

accounts for approximately two-thirds of the variance in final first-innings totals across the test set, demonstrating that the natural-language prompt encodes substantial predictive signal that the fine-tuned model successfully exploits.

4.4 Zero-Shot Frontier Model: GPT-4.1-nano

The *GPT-4.1-nano* model was evaluated in a zero-shot setting using the summary prompt variant (without the trailing "Final 1st innings score:" suffix) and an explicit system instruction to respond with the predicted score as a number only. The model was accessed via the LiteLLM library and evaluated on 200 test datapoints from the temporal test split, drawn from the same underlying match instances as the prompt-format test split used for the Llama evaluations.

The results are summarised in Table 4.3.

Table 4.3: *GPT-4.1-nano* zero-shot evaluation results (200 datapoints)

Model	**MAE (runs)**	**MSE**	R^2 **(%)**
GPT-4.1-nano (zero-shot)	30.02	1,538	14.0

Despite being a substantially larger and more capable model by general-purpose benchmarks, *GPT-4.1-nano* achieves poor predictive performance on this task, with an MAE of **30.02 runs** and an R^2 of only **14.0%**. The low R^2 indicates that the model's predictions account for only a small fraction of the variance in first-innings totals, performing only marginally better than a naïve mean predictor.

A qualitative inspection of the model's outputs revealed a tendency to underestimate scores in early-innings states, where fewer overs have been bowled and the cumulative run tally is low. For example, at a venue with a par score of 160, after 2 overs with 14 runs scored and 1 wicket down, the model predicted a final total of 89 runs against an actual final score of 160. This pattern suggests that the frontier model does not reliably scale early-innings run rates to final totals, and that its broad pre-training does not encode sufficient domain-specific knowledge about T20 scoring dynamics to compensate for the absence of task-specific fine-tuning.

4.5 Cross-Condition Comparison

Table 4.4 presents the consolidated performance of all evaluated conditions. The fine-tuned *Llama 3.2-3B* model (step-800 checkpoint, 500 datapoints) is used as the representative result for the fine-tuned condition, and the *GPT-4.1-nano* result at 200 datapoints is retained as reported (a 500-point evaluation was not conducted for the frontier model due to API cost constraints).

Table 4.4: Cross-condition performance comparison on the temporal test set

Model	**Fine-tuned**	**N**	**MAE (runs)**	**MSE**	R^2 **(%)**
Llama 3.2-3B (zero-shot)	No	200	71.31	7,463	-317.4
Llama 3.2-3B (fine-tuned, step-800)	Yes	500	**16.51**	**543**	**68.3**
GPT-4.1-nano (zero-shot)	No	200	30.02	1,538	14.0

Note: The fine-tuned model is evaluated at $N = 500$ to reduce evaluation noise; the zero-shot Llama baseline and GPT-4.1-nano are evaluated at $N = 200$. All evaluation sets comprise the chronologically first N rows of the temporal test split.

The results reveal three principal findings.

Finding 1 — Fine-tuning substantially outperforms the frontier zero-shot model. The fine-tuned *Llama 3.2-3B* model achieves an MAE of 16.51 runs and an R^2 of 68.3%, compared to an MAE of 30.02 runs and an R^2 of 14.0% for zero-shot *GPT-4.1-nano*. This represents a **45.0% reduction in MAE** and a **54.3 percentage-point improvement in** R^2 relative to the frontier model, despite the fine-tuned model being a compact open-source system compared to the closed proprietary frontier model. The result demonstrates that domain-specific supervised fine-tuning on a curated T20 innings dataset is far more effective than relying on the general world knowledge of a large proprietary model for this task.

Finding 2 — Fine-tuning produces a dramatic improvement over the zero-shot Llama baseline. The zero-shot *Llama 3.2-3B* baseline achieves an MAE of 71.31 runs and an R^2 of -317.4%, indicating near-random prediction behaviour. Fine-tuning the same model on the domain-specific dataset reduces the MAE to 16.51 runs — a **76.8% reduction** — and raises R^2 from -317.4% to 68.3%, crossing the zero threshold from a below-mean

predictor to a model that accounts for more than two-thirds of the variance in final first-innings totals. This dramatic gain demonstrates that the task knowledge required for in-game T20 score prediction is not present in the base pre-trained weights and must be acquired through domain-specific supervised training.

Finding 3 — Checkpoint selection has a marginal but consistent effect at scale. The step-800 minimum-loss checkpoint outperforms the final epoch checkpoint on all three metrics when evaluated over 500 datapoints, confirming the validity of early-stopping-based checkpoint selection. The difference is small in absolute terms (MAE: 16.51 vs. 16.54 runs; R^2: 68.3% vs. 68.1%), but is consistent across all metrics, supporting the selection of the step-800 checkpoint as the deployment model.

Chapter 5

Discussion

5.1 Introduction

This chapter interprets the experimental results presented in Chapter 4 in the context of the research objectives stated in Section 1.4. Section 5.2 discusses the effectiveness of fine-tuning relative to both the zero-shot Llama baseline and the frontier model (Research Objectives 3, 4, and 6). Section 5.3 examines the role of the engineered features and prompt schema in enabling model learning (Research Objectives 1 and 2). Section 5.4 interprets the checkpoint selection findings (Research Objective 5). Section 5.5 positions the results relative to the existing literature. Section 5.6 discusses the limitations of the study, and Section 5.7 proposes directions for future research.

5.2 Effectiveness of Fine-Tuning for Score Prediction

The most prominent finding of this study is the large and consistent performance gap between the fine-tuned *Llama 3.2-3B* model and the two zero-shot baselines. The fine-tuned model achieves an MAE of 16.51 runs and an R^2 of 68.3%, whereas the zero-shot *Llama 3.2-3B* baseline achieves an MAE of 71.31 runs and an R^2 of -317.4%. This 76.8% reduction in MAE and the shift from strongly negative to positive R^2 upon fine-tuning makes clear that the base pre-trained model possesses no useful prior knowledge for this task in the zero-shot setting.

This outcome is consistent with the theoretical motivation for supervised fine-tuning of LLMs on specialised tasks [23]. Cricket scoring dynamics — especially the non-linear relationship between early-innings run rate, wickets in hand, over count, venue character-

istics, and final total — are not the subject of large-scale structured training data in general pre-training corpora. Prose commentary, match reports, and encyclopaedic articles about cricket are abundant on the internet, but they do not encode the kind of quantitative, conditional distribution that a model needs to learn for score prediction from a mid-innings state. Fine-tuning on approximately 24,080 structured prompt–completion pairs (drawn from 2,408 training matches at ten snapshots per innings) of the form "here is the current match state; the final score was X" directly provides this mapping.

The comparison with *GPT-4.1-nano* is equally illuminating. Despite *GPT-4.1-nano* being a frontier proprietary model trained on a substantially larger corpus, it achieves an MAE of 30.02 runs and an R^2 of only 14.0% — substantially worse than the fine-tuned model. This supports a key premise of the study: that scale alone does not compensate for the absence of domain-specific fine-tuning on structured quantitative data. The qualitative observation that *GPT-4.1-nano* tends to underestimate final totals from early-innings states — for example, predicting 89 runs when 14 runs have been scored in 2 overs at a high-scoring venue — suggests that the frontier model applies a rough proportional scaling heuristic ("14 runs in 2 overs projects to around 90 runs in 20 overs") without accounting for the empirically observed acceleration in run-scoring that characterises T20 cricket. In contrast, the fine-tuned model has learned from thousands of examples in which a 7-run-per-over early rate at a 160-par venue results in a final total well above the raw projection.

5.3 Role of Feature Engineering and Prompt Schema

The strong performance of the fine-tuned model cannot be attributed to fine-tuning alone. A key design decision in this work is the translation of raw ball-by-ball match data into a richly contextualised natural-language prompt that encodes features not recoverable from a bare score-and-wickets description. Three engineered features are particularly significant.

Venue par score normalises the prediction target relative to the ground's historical scoring environment. A model trained without this feature would need to learn venue-specific

scoring rates implicitly from the venue name, which is a substantially harder problem. By encoding the par score directly in the prompt (e.g., "venue par score: 162"), the model is given an explicit reference point around which to calibrate its prediction.

Remaining Batting Strength Score (RBSS) captures the batting depth remaining beyond the current over, weighted by the expected contributions of the remaining batters' roles. Without this feature, two match states with identical over-counts and scores but very different batting line-ups would be indistinguishable to the model. The inclusion of RBSS allows the model to infer, for example, that a tail-heavy batting lineup is unlikely to sustain the current scoring rate into the final overs.

Player classification assigns each batting team member to one of five roles (`BATTER`, `ALL_ROUNDER`, `UTILITY_PLAYER`, `BOWLER`, or `INSUFFICIENT_DATA`), providing the basis for the RBSS computation. By encoding the aggregate batting quality of the remaining lineup as a single scalar, the model can infer, for example, that a tail-heavy batting order is unlikely to sustain the current scoring rate into the final overs.

The verbatim prompt template was designed to be both human-readable and machine-parseable, allowing the LLM to directly associate the structured field values with the prediction target during supervised fine-tuning. The segment-based construction of one prompt per two-over section, rather than per match, amplifies the effective training set size and presents the model with diverse in-progress states from across each innings.

5.4 Checkpoint Selection and Generalisation

The checkpoint selection analysis provides empirical support for early-stopping based on minimum validation loss as the preferred model selection criterion [26]. At 200 test datapoints, the final epoch checkpoint appears to outperform step-800 (MAE 16.12 vs. 17.05), but the MSE comparison contradicts this ordering (601 vs. 618), indicating that the 200-point evaluation is subject to sample noise. At 500 datapoints, the step-800 checkpoint outperforms the final checkpoint on all three metrics consistently, albeit by a narrow margin.

This pattern is characteristic of mild overfitting: continued training beyond the validation-loss minimum improves performance on the training distribution but introduces a small generalisation penalty on the unseen test set. In the context of this study, the effect is modest — the difference between checkpoints is less than 0.03 runs in MAE at 500 datapoints — because QLoRA fine-tuning with low-rank adapters is inherently regularised. Full fine-tuning of all model weights would be expected to exhibit a larger and earlier divergence between the minimum-validation-loss checkpoint and the final checkpoint.

The reversal of checkpoint ordering between the 200- and 500-point evaluations also highlights the practical importance of test-set size in evaluating generative prediction models. A 200-point temporally contiguous evaluation window can be dominated by a cluster of matches at a particular venue or tournament stage, distorting the apparent relative performance of two otherwise comparable models. The larger 500-point evaluation is more representative of the full test distribution and therefore a more reliable basis for model selection.

It should also be noted that the validation loss used to select checkpoints during training was computed over the chronologically first 50 rows of the validation split rather than the full 2,370-row validation set, a choice necessitated by GPU time constraints on the Kaggle T4 platform. The resulting loss estimates carry a higher standard error than a full-split evaluation, introducing a degree of noise into the checkpoint ordering. That the step-800 checkpoint nonetheless demonstrates consistent superiority across all three metrics at 500 test datapoints suggests that the minimum-loss signal from the 50-row subsample was sufficient to identify the better-generalising checkpoint in this instance, though this cannot be guaranteed in general.

5.5 Positioning Against the Literature

The MAE of 16.51 runs achieved by the fine-tuned model, evaluated across all ten innings sections from the opening two-over segment through to the final, is notable given that the prediction task is posed from the earliest possible innings states where uncertainty is great-

est. Most prior work on score or outcome prediction in T20 cricket restricts evaluation to later innings stages where remaining variance is lower and absolute error figures are correspondingly smaller [17, 18]; direct numerical comparison with the present aggregate results is therefore not meaningful without matched evaluation ranges.

Regression-based and Markov chain approaches [3, 25] operate over explicitly modelled ball-outcome transition probabilities that must be estimated per venue and per match-up. The present approach requires no such hand-crafted probabilistic model: the LLM learns the scoring distribution implicitly from the fine-tuning data, with domain knowledge injected through engineered features.

The broader literature on LLMs applied to numerical and tabular prediction tasks has shown that fine-tuned small LLMs can match or exceed larger zero-shot models on domain-specific tasks when provided with structured, informative prompts [13, 11]. The results of this study are consistent with this emerging finding, and extend it to a real-world sports analytics setting characterised by high within-match stochasticity.

5.6 Limitations

Several limitations of the present study should be acknowledged.

Match data scope. The training and evaluation data are restricted to international men's T20 matches sourced from Cricsheet. Domestic T20 leagues (Indian Premier League, Big Bash League, Pakistan Super League, and others) account for the majority of professional T20 cricket played globally and may exhibit markedly different scoring dynamics due to ground dimensions, match conditions, and player composition. The model trained in this study cannot be assumed to generalise to domestic league contexts without retraining on league-specific data.

Quantisation and hardware. Both the zero-shot baseline and the fine-tuned model were evaluated using 4-bit NF4 quantisation on a Kaggle T4 GPU, which is a consumer-grade accelerator. Full-precision inference on a larger accelerator could yield modestly different predictions. Similarly, the fine-tuning was performed with QLoRA adapters rather than

full fine-tuning of all model weights. Full fine-tuning on a larger GPU cluster may yield further performance improvements, though at substantially higher computational cost.

Base model size. The study uses *Llama 3.2-3B* as the base model. Larger open-source models (e.g., *Llama 3.2-8B* or *Llama 3.1-70B*) may achieve better performance in both the zero-shot and fine-tuned settings. The choice of a 3B model was governed by the available Kaggle T4 GPU memory budget. The relative improvement from fine-tuning may differ for larger base models.

Evaluation on a single fine-tuning run. The fine-tuned model results are from a single training run on Kaggle. Variation in model performance across multiple fine-tuning runs with different random seeds has not been quantified. The reported results should therefore be interpreted as point estimates rather than stable averages.

Zero-shot Llama re-run. The zero-shot *Llama 3.2-3B* evaluation was conducted on Google Colab in a single session. While the results are internally consistent and align with theoretical expectations, independent replication of the zero-shot baseline was not performed.

Temporal test split and distribution shift. The temporal test split ensures that test matches postdate all training matches, providing a realistic evaluation scenario. However, international cricket schedules are not uniformly distributed in time: test matches may cluster around major tournaments, which could introduce venue and team-composition biases not present in the training data. This risk was partially mitigated by the large size of the test set (up to 500 datapoints), but a stratified analysis by tournament or time period was beyond the scope of the present work.

5.7 Future Work

Several directions for future research follow naturally from the present findings.

Extension to domestic T20 leagues. The most immediately impactful extension would be to incorporate ball-by-ball data from major domestic leagues into the training pipeline. The Cricsheet archive includes IPL, BBL, and PSL data that could be processed through

the same feature-engineering and prompt-generation pipeline developed in this work, enabling a model trained on a substantially broader and more commercially relevant distribution of matches.

Larger and more recent base models. Testing the fine-tuning pipeline with more recent and larger open-source models — such as *Llama 3.1-8B* or *Mistral-7B-Instruct* — would determine whether the performance gains from fine-tuning are robust to the choice of base model and whether larger models exhibit steeper improvement curves with the same training data volume.

Second-innings prediction. The current study is restricted to first-innings prediction. Extending the framework to second-innings prediction, where the target (winning or losing by a given margin) is a dynamic chase target rather than an open final total, would substantially broaden the practical applicability of the approach. Second-innings states also provide richer contextual features (current chase target, required run rate, resources remaining) that may be well-suited to natural-language prompt encoding.

Live deployment and real-time inference. The inference pipeline developed in this study evaluates pre-generated prompts from stored match data. Integrating the pipeline with a live data feed — such as the ESPN Cricinfo API or Cricsheet's near-real-time updates — would enable evaluation of the model in a true live prediction setting, where the prompt is generated from ball-by-ball updates as the innings progresses.

Reinforcement learning from human feedback. The current fine-tuning objective is supervised regression over the predicted final score. An alternative approach would be to frame the prediction task as a generation problem optimised with direct preference optimisation (DPO) [28] or a custom reward signal that penalises large errors more strongly than small ones, potentially improving calibration in high-variance early-innings states.

Uncertainty quantification. The model currently produces a single point prediction. Generating a prediction interval or probability distribution over final scores — for example, by sampling multiple completions with non-zero temperature and summarising the distribution — would be more informative for downstream applications such as in-

play betting markets or broadcast commentary tools, where quantification of prediction uncertainty is valuable.

Chapter 6

Conclusions

6.1 Summary of the Study

This thesis investigated whether a small, open-source large language model could be adapted for the task of first-innings score prediction in T20 cricket through parameter-efficient fine-tuning on domain-specific data. The central argument was that the structured, context-rich nature of in-game cricket match states could be encoded as natural-language prompts, enabling a fine-tuned LLM to learn the mapping from mid-innings conditions to final scores without the need for an explicit probabilistic model of ball outcomes.

To test this argument, a complete pipeline was constructed: raw ball-by-ball match data from the Cricsheet archive was preprocessed and enriched with engineered features, including a venue par score derived from historical ground averages, a Remaining Batting Strength Score (RBSS) that quantifies remaining batting depth as a weighted sum of player role contributions, and player classifications for batting team members. These features were serialised into a structured natural-language prompt schema, producing one training example per two-over section per innings. After temporal filtering, approximately 24,080 training examples covering approximately 2,408 international men's T20 matches were used to fine-tune *Llama 3.2-3B* using QLoRA (4-bit NF4 quantisation with rank-16 LoRA adapters) on a Kaggle T4 GPU. The fine-tuned model was then evaluated on a held-out temporal test set alongside two zero-shot baselines: the unmodified *Llama 3.2-3B* base model and the frontier model *GPT-4.1-nano*.

6.2 Principal Findings

The experiments produced three principal findings.

First, domain-specific fine-tuning transforms an LLM with no useful prior knowledge for this task into a competitive score predictor. The unmodified *Llama 3.2-3B* model achieves an MAE of 71.31 runs and an R^2 of -317.4% in the zero-shot setting, confirming that general pre-training provides essentially no usable knowledge for structured numerical prediction from mid-innings cricket states. After fine-tuning on approximately 24,080 domain-specific examples, the same model achieves an MAE of 16.51 runs and an R^2 of 68.3% — a 76.8% reduction in MAE and a shift from a strongly negative R^2 to 68.3%, crossing the threshold from a below-mean predictor to a model that explains more than two-thirds of the variance in final score.

Second, a small fine-tuned model substantially outperforms a much larger frontier model in the zero-shot setting. *GPT-4.1-nano* achieves an MAE of 30.02 runs and an R^2 of 14.0% without fine-tuning — considerably worse than the fine-tuned model on both metrics. This result demonstrates that scale alone does not substitute for domain-specific supervised training on structured quantitative data, and that parameter-efficient fine-tuning of a small open-source model is a viable and cost-effective alternative to prompt-engineering a large proprietary frontier model for specialised prediction tasks.

Third, checkpoint selection based on minimum validation loss provides a marginal but consistent generalisation benefit. The step-800 checkpoint, which corresponds to the minimum evaluation loss during training, outperforms the final epoch checkpoint on all three metrics when evaluated on 500 test datapoints (MAE: 16.51 vs. 16.54 runs; R^2: 68.3% vs. 68.1%), confirming the standard recommendation to prefer minimum-validation-loss checkpoints over final-epoch checkpoints in fine-tuning workflows.

6.3 Contributions

This study makes three primary contributions to the fields of sports analytics and applied natural language processing.

Contribution 1 — A complete, reproducible T20 score prediction pipeline. The end-to-end pipeline from raw Cricsheet JSON data through feature engineering, prompt generation, QLoRA fine-tuning, and evaluation is fully documented and versioned, providing a reproducible baseline for future work in LLM-based cricket analytics.

Contribution 2 — The Remaining Batting Strength Score (RBSS) feature. The RBSS is a novel engineered feature that quantifies the expected run-scoring potential of the batting depth remaining in an innings, weighted by the assigned role of each player. This feature provides the model with information about the batting lineup that is not recoverable from the current score and wickets-in-hand alone, and its inclusion in the prompt schema is a key factor in enabling the model to learn two-over section score dynamics.

Contribution 3 — Empirical evidence for fine-tuned LLM superiority over frontier zero-shot models on domain-specific numerical prediction. The comparison between fine-tuned *Llama 3.2-3B* and zero-shot *GPT-4.1-nano* provides a concrete, quantitative illustration of a principle that has been observed in the broader LLM literature but has not previously been demonstrated in a cricket analytics context: that domain-specific fine-tuning of a small open-source model yields substantially better results than zero-shot prompting of a large frontier model for structured numerical prediction tasks.

6.4 Recommendations for Practitioners

Based on the findings of this study, the following practical recommendations are offered to data scientists and machine learning engineers working on sports prediction problems. When the prediction target is a structured numerical outcome derived from a well-defined in-game state (such as a cricket final total, a basketball final score, or a football goal tally), feature engineering to encode domain knowledge in the prompt is at least as important

as model selection. A well-engineered prompt schema that includes venue context, remaining batting resources, and opponent quality will enable a small fine-tuned model to outperform a frontier model receiving only raw in-game statistics.

For prediction tasks involving high stochasticity (e.g., early-innings score prediction where fewer than 5 overs have been bowled), point prediction from a single model completion is likely to produce poorly calibrated estimates. Practitioners are encouraged to report prediction intervals or to evaluate multiple sampled completions in order to quantify uncertainty.

For fine-tuning workflows with limited GPU memory, QLoRA with rank-16 adapters and 4-bit NF4 quantisation provides an effective trade-off between model quality and memory footprint, and the minimum-validation-loss checkpoint should be adopted in preference to the final epoch checkpoint.

6.5 Concluding Remarks

This thesis demonstrates that context-aware fine-tuning of large language models on domain-specific, structured natural-language prompts is a viable and effective approach for first-innings score prediction in T20 cricket. The work makes a case for the broader applicability of parameter-efficient fine-tuning as a practical methodology for sports analytics: one that leverages the representational power of pre-trained language models while remaining tractable on consumer-grade hardware. The pipeline, features, and experimental framework developed here provide a foundation on which future work can build — extending to domestic leagues, larger models, second-innings prediction, and live real-time deployment.

Chapter 7

Recommendations

7.1 Introduction

This chapter consolidates actionable recommendations arising from the study, organised by stakeholder group. The recommendations are directed at four distinct audiences: cricket governing bodies and team analytics units who may wish to deploy predictive tools in operational settings; machine learning researchers extending this line of inquiry; software engineers and MLOps practitioners responsible for productionising fine-tuned language models; and future academic researchers building on the dataset, pipeline, or methodology developed in this work.

7.2 Recommendations for Cricket Boards and Analytics Teams

7.2.1 Adopt Domain-Specific Fine-Tuning Over Frontier Model Prompting

The empirical results of this study demonstrate that a fine-tuned 3-billion-parameter open-source model achieves substantially better predictive accuracy (MAE 16.51 runs, R^2 68.3%) than a zero-shot frontier model (MAE 30.02 runs, R^2 14.0%). Cricket analytics teams should therefore prioritise investment in curating labelled training data and running parameter-efficient fine-tuning over subscription-based access to general-purpose frontier models for specialised prediction tasks. The computational cost of QLoRA fine-tuning on a single consumer-grade GPU is modest — less than six hours on a Kaggle T4 — and the resulting model can be hosted on-premises without ongoing API fees.

7.2.2 Invest in Structured Ball-by-Ball Data Infrastructure

The quality of score prediction models is fundamentally constrained by the quality and quantity of the underlying ball-by-ball data. Teams and national boards that maintain private performance databases should consider standardising their internal data formats to be compatible with openly available archives such as Cricsheet, enabling rapid integration of new data into fine-tuning pipelines. In particular, accurate recording of player roles (specialist batter, all-rounder, specialist bowler) is essential for computing the Remaining Batting Strength Score (RBSS) feature introduced in this work.

7.2.3 Use Venue Par Scores as a Baseline Calibration Reference

The venue par score feature, derived from the historical average first-innings total at each ground, was identified as a key contextual signal in the prompt schema. Analytics teams are encouraged to maintain and regularly update venue par score tables for all grounds at which they compete, distinguishing between day and day/night matches where sufficient data are available. These par scores can also serve as a standalone heuristic for early-innings score targets in the absence of a full predictive model.

7.2.4 Do Not Use Point Predictions Alone for High-Stakes Decisions

A model with an average prediction error of approximately 16 runs — roughly 10% of a typical T20 total — is useful for trend analysis and strategy simulation, but should not be used in isolation for high-stakes decisions such as Duckworth-Lewis target setting or in-play betting market settlement. For such applications, a prediction interval or a Monte Carlo simulation over multiple sampled completions is preferable to a single greedy-decoded prediction.

7.3 Recommendations for Machine Learning Researchers

7.3.1 Prioritise Feature Engineering for Structured Tabular-to-Text Tasks

The results of this study illustrate a general principle applicable beyond cricket: when adapting an LLM to a structured numerical prediction task, the information content of the prompt is as important as the choice of base model or fine-tuning configuration. Researchers applying LLMs to analogous domains — energy consumption forecasting, financial time-series, clinical outcome prediction — should devote equivalent attention to prompt schema design and domain-specific feature engineering as to hyperparameter search and model selection.

7.3.2 Evaluate at Multiple Test-Set Sizes Before Drawing Conclusions

The checkpoint comparison in this study revealed that results at 200 test datapoints partially reversed at 500 datapoints, with the apparent ranking of the two checkpoints depending on which metric was prioritised. Researchers evaluating generative prediction models on temporally ordered test sets should report results at multiple test-set sizes and verify that rankings are consistent before drawing conclusions about the relative performance of model variants. Where possible, bootstrapped confidence intervals over the evaluation metrics should be computed and reported.

7.3.3 Consider the Evaluation Impact of Regex Post-Processing

The post-processing step that extracts a numeric prediction from each model completion is a non-trivial component of the evaluation pipeline. When a model fails to generate a recognisable integer in its completion, the fallback value of 0 is used, which substantially inflates the MAE and MSE for that prediction. Researchers should report the rate of failed regex extractions alongside the aggregate metrics, as a high failure rate indicates that the model has not converged to the expected output format rather than that it is generating poor predictions. For the zero-shot baseline in this study, the high failure rate was a primary

contributor to the extreme MAE of 71.31 runs.

7.3.4 Report Results on Both Checkpoint Variants

When fine-tuning LLMs for regression or numerical prediction tasks, researchers should report results for both the minimum-validation-loss checkpoint and the final epoch checkpoint separately. Reporting only the final checkpoint may understate model performance, while reporting only the minimum-loss checkpoint may obscure whether the training process has converged. The present study found a consistent but small advantage for the minimum-loss checkpoint; other tasks and training configurations may exhibit larger divergences.

7.4 Recommendations for Software Engineers and MLOps Practitioners

7.4.1 Use QLoRA for Memory-Constrained Deployment

QLoRA (4-bit NF4 quantisation with low-rank LoRA adapters) provides an effective method for fine-tuning and deploying language models on GPU hardware with limited memory. The *Llama 3.2-3B* model with rank-16 LoRA adapters and 4-bit weights fits comfortably within the 15 GB VRAM of a Kaggle T4 or Google Colab T4 instance during both training and inference. Practitioners deploying fine-tuned models in resource-constrained environments are encouraged to adopt this configuration as a default starting point.

7.4.2 Implement Checkpoint Management from the Start of Training

The Weights & Biases integration used in this study made it straightforward to identify the step-800 minimum-loss checkpoint after training. Practitioners should configure checkpoint saving at regular intervals (every 200–400 steps for a training run of this scale) from the beginning of the training job, and should log validation loss at each checkpoint. At-

tempting to recover intermediate checkpoints after the fact is not possible if periodic saving was not enabled in advance.

7.4.3 Wrap Inference in a Robust Post-Processing Layer

Generative LLM outputs are inherently unstructured and may deviate from the expected format even after fine-tuning on uniformly formatted training data. The two-step regex post-processing pipeline described in Section 3.6.3 — which first attempts to match "Final 1st innings score: *N*" and falls back to the first standalone integer in the completion — should be treated as a minimum viable post-processing layer. Production deployments should additionally validate that the extracted integer falls within a plausible range (e.g., 50–350 runs for a T20 match), log any out-of-range or failed extractions for monitoring, and alert operators when the failure rate exceeds a configurable threshold.

7.4.4 Version Datasets and Model Artefacts Together

The fine-tuned model artefact (Hugging Face model repository) and the training dataset (Hugging Face dataset repository) used in this study were versioned independently. For reproducibility in production systems, model artefacts should be tagged with the exact dataset version and training configuration that produced them. This is particularly important for iterative fine-tuning workflows in which the training data is periodically refreshed with new match data, as a model trained on a later dataset version may not be directly comparable to one trained on an earlier version.

7.4.5 Use Serverless GPU Platforms as a Low-Cost Deployment Path

The deployment experiment described in Section 3.7 demonstrated that a layered inference stack — Modal serverless GPU backend, Hugging Face Spaces prototype, and a static React dashboard — can be assembled from off-the-shelf cloud components with no persistent infrastructure and no upfront cost. A 4-bit QLoRA model can be deployed on a T4 container with a cold start latency of approximately 30 seconds and a warm response latency

of approximately 2 seconds, at zero compute cost when idle (scale-to-zero configuration). Two deployment patterns are worth distinguishing: an *ephemeral function*, which reloads the model on every call and is appropriate for infrequent one-off predictions; and a *persistent warm service*, which loads the model once per container lifetime and amortises that cost across repeated calls. The warm service pattern, with model weights cached in a persistent volume, is recommended for any production scenario that requires sub-five-second response times.

For the user-facing tier, two approaches were prototyped. The Gradio [1] interface hosted on Hugging Face Spaces [16] is the faster path to a shareable URL: it requires no front-end development expertise and incurs no compute charge at idle, making it well-suited for research demonstrations and stakeholder review. For a broadcast-quality application, a React single-page application built with Vite and Tailwind CSS provides a richer and more customisable experience, can be deployed as a static bundle to Azure Static Web Apps via a GitHub Actions CI/CD pipeline backed by infrastructure-as-code, and decouples the UI from the backend through a clean REST API contract (`POST /api/predict`). Crucially, the React application includes a local heuristic mock mode so the full interface can be demonstrated without a live inference backend. Practitioners should adopt the Gradio prototype first to validate the product concept, and migrate to the React dashboard when broadcast-quality presentation or custom interactivity is required.

7.5 Recommendations for Future Researchers

7.5.1 Extend the Pipeline to Domestic T20 Leagues

The pipeline developed in this study processes Cricsheet JSON archives and is format-agnostic with respect to competition type. The most impactful near-term extension is to include ball-by-ball data from major domestic leagues (IPL, BBL, PSL, CPL, SA20) in the training set. Since these leagues collectively produce more professional T20 cricket than international fixtures, a model trained on a combined dataset would be applicable to

a far broader range of matches and scoring environments. The Cricsheet archive includes this data and requires no additional preprocessing to integrate.

7.5.2 Investigate the Effect of Base Model Scale

This study used *Llama 3.2-3B* as the base model, driven by GPU memory constraints. Future work should systematically evaluate the performance of the same QLoRA fine-tuning pipeline applied to larger base models (7B, 8B, and 70B parameter variants) to determine whether the predictive accuracy scales with model size and whether the relative improvement from fine-tuning changes at larger scales. It is possible that larger base models exhibit stronger zero-shot performance due to better general numerical reasoning, which would reduce the marginal gain from fine-tuning.

7.5.3 Develop a Second-Innings Prediction Variant

A second-innings prediction model would need to condition on the first-innings total as an additional input and predict the probability of a successful chase rather than (or in addition to) the final second-innings score. This is a more complex prediction target due to the strategic interaction between batting and field-setting, but it is commercially and analytically more valuable for live match commentary and in-play applications. The prompt schema developed in this work could be extended to include the chase target and the required run rate as additional features.

7.5.4 Quantify Uncertainty with Sampled Completions

The current evaluation uses greedy decoding to generate a single point prediction per prompt. Future work should evaluate the use of nucleus sampling (top-p sampling) to generate an ensemble of predictions from each prompt and derive a prediction interval. This would allow calibration analysis (e.g., whether the 90% prediction interval contains the actual score 90% of the time) and would make the model more useful for applications requiring uncertainty estimates, such as real-time broadcast analytics and in-play risk man-

agement.

7.5.5 Conduct an Ablation Study of Engineered Features

The prompt schema used in this study combines six categories of engineered features: over count and segment label, current score and wickets, recent scoring rate and boundary counts, venue par score, RBSS, and active player classifications. The relative contribution of each feature to model performance has not been isolated. A systematic ablation study — in which each feature is individually removed from the prompt and the model is re-evaluated — would clarify which features are most informative and whether any features are redundant or counterproductive. This analysis would also guide feature selection for future extensions of the pipeline.

Bibliography

[1] Abid, A., Abdalla, A., Abid, A., Khan, D., Alfozan, A. and Zou, J. (2019) Gradio: Hassle-free sharing and testing of ML models in the wild. *arXiv preprint arXiv:1906.02569*.

[2] Ahmad, H., Daud, A., Wang, L., Hong, H., Dawood, H. and Ahmad, W. (2011) Prediction of winning team in one day international cricket matches using machine learning. *Journal of Computers*, 6(5), 957–964.

[3] Bailey, M. and Clarke, S.R. (2004) Predicting the match outcome in one day international cricket matches, while the match is in progress. *Journal of Sports Science and Medicine*, 3(1), 16–24.

[4] BerriAI (2023) *LiteLLM: Call all LLM APIs using the OpenAI format* [online]. Available at: `https://github.com/BerriAI/litellm` (Accessed: 21 April 2026).

[5] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J.D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G. and Askell, A. et al. (2020) Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.

[6] Cricsheet (2024) *Cricsheet: A cricketing archive* [online]. Available at: `https://cricsheet.org` (Accessed: 18 April 2026).

[7] Davis, J., Perera, H. and Swartz, T.B. (2015) Player evaluation in Twenty20 cricket. *Journal of Sports Analytics*, 1(1), 19–31.

[8] Dettmers, T., Pagnoni, A., Holtzman, A. and Zettlemoyer, L. (2023) QLoRA: Efficient finetuning of quantized LLMs. *Advances in Neural Information Processing Systems*, 36, 10088–10115.

[9] Devlin, J., Chang, M.W., Lee, K. and Toutanova, K. (2019) BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*, 4171–4186.

[10] Duckworth, F.C. and Lewis, A.J. (1998) A fair method for resetting the target in interrupted one-day cricket matches. *Journal of the Operational Research Society*, 49(3), 220–227.

[11] Gruver, N., Finzi, M., Qiu, S. and Wilson, A.G. (2023) Large language models are zero-shot time series forecasters. *Advances in Neural Information Processing Systems*, 36.

[12] Haghighat, M., Rastegari, H. and Nourafza, N. (2013) A review of data mining techniques for result prediction in sports. *ACSIJ Advances in Computer Science: an International Journal*, 2(5), 7–12.

[13] Hegselmann, S., Buendia, A., Lang, H., Agrawal, M., Jiang, X. and Sontag, D. (2023) TabLLM: Few-shot classification of tabular data with large language models. *Proceedings of the 26th International Conference on Artificial Intelligence and Statistics*, 5549–5581.

[14] Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L. and Chen, W. (2022) LoRA: Low-rank adaptation of large language models. *International Conference on Learning Representations*.

[15] Huckerby, J., Whitworth, A. and Dhaliwal, R. (2024) Leveraging large language models for football match outcome prediction. *Machine Learning and Data Mining for Sports Analytics Workshop, ECML PKDD*.

[16] Hugging Face (2024) *Hugging Face Spaces* [online]. Available at: `https://huggingface.co/spaces` (Accessed: 16 May 2026).

[17] Iyer, S.R. and Sharda, R. (2009) Prediction of athletes' performance using neural networks: An application in cricket team selection. *Expert Systems with Applications*, 36(3), 5510–5522.

[18] Kampakis, S. and Thomas, W. (2015) Using machine learning to predict the outcome of English county twenty over cricket matches. *Journal of Sports Analytics*, 1(1), 63–76.

[19] Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A.A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., Hassabis, D., Clopath, C., Kumaran, D. and Hadsell, R. (2017) Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13), 3521–3526.

[20] Loshchilov, I. and Hutter, F. (2019) Decoupled weight decay regularization. *International Conference on Learning Representations*.

[21] Meta AI (2024) The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.

[22] Modal Labs (2024) *Modal: Serverless Cloud for AI and ML* [online]. Available at: `https://modal.com` (Accessed: 18 April 2026).

[23] Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K. and Ray, A. et al. (2022) Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35, 27730–27744.

[24] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. and Duchesnay, E. (2011) Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[25] Perera, H. and Swartz, T.B. (2012) Resource estimation in T20 cricket. *IMA Journal of Management Mathematics*, 24(3), 337–347.

[26] Prechelt, L. (1998) Early stopping — but when? In: Orr, G.B. and Müller, K.R. (eds.) *Neural Networks: Tricks of the Trade*. Berlin: Springer, pp. 55–69.

[27] Preston, I. and Thomas, J. (2000) Batting strategy in limited overs cricket. *Journal of the Royal Statistical Society: Series D (The Statistician)*, 49(1), 95–106.

[28] Rafailov, R., Sharma, A., Mitchell, E., Manning, C.D., Ermon, S. and Finn, C. (2023) Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36.

[29] Sankaranarayanan, V.V., Sattar, J. and Lakshminarayanan, V. (2014) Auto-play: A data mining approach to ODI cricket simulation and prediction. *Proceedings of the SIAM International Conference on Data Mining*, 1064–1072.

[30] Stern, S.E. (2016) The Duckworth-Lewis-Stern method: Extending the Duckworth-Lewis methodology to deal with modern scoring rates. *Journal of the Operational Research Society*, 67(12), 1469–1480.

[31] Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E. and Azhar, F. et al. (2023) LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.

[32] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł. and Polosukhin, I. (2017) Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998–6008.

[33] von Werra, L., Belkada, Y., Tunstall, L., Beeching, E., Thrush, T. and Lambert, N. (2022) *TRL: Transformer Reinforcement Learning* [online]. Available at: `https://github.com/huggingface/trl` (Accessed: 18 April 2026).

[34] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Chi, E., Le, Q. and Zhou, D. (2022) Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35, 24824–24837.

Chapter A

Prompt Schema and Sample Prompt–Completion Pairs

A.1 Full Prompt Template

Each in-game state snapshot is serialised into the following structured natural-language prompt. Placeholders in braces are substituted with the computed feature values at the end of the relevant two-over section.

```
Match type: T20
Venue: {venue}
Venue par score: {par_score_filled}
Batting team: {team}

Overs completed: {overs_completed}
Balls remaining: {balls_remaining}
Runs scored: {cumulative_runs}
Wickets lost: {cumulative_wickets}
Current run rate: {cumulative_run_rate}

Runs in last 2 overs: {runs_in_section}
Dot balls so far: {dot_balls_so_far}
Dot balls in last 2 overs: {dot_balls_last_2_overs}
```

```
Fours hit: {fours_hit}
Sixes hit: {sixes_hit}

Powerplay completed: {Yes|No}

Remaining Batting Strength Score: {RBSS}

Final 1st innings score:
```

The completion (target output) appended during fine-tuning and expected during inference is the actual final first-innings total as a plain integer string, for example `173`.

A.2 Sample Prompt–Completion Pairs

The following three examples are drawn from the test split and illustrate how the prompt encodes progressively later over-boundary states within the same innings. Section labels (Early, Middle, Death) are added here for clarity and do not appear in the actual prompt.

Example 1 — Early-innings snapshot (after 4 overs, Section 2)

```
Match type: T20
Venue: SuperSport Park
Venue par score: 168
Batting team: South Africa

Overs completed: 4
Balls remaining: 96
Runs scored: 38
Wickets lost: 1
Current run rate: 9.50
```

```
Runs in last 2 overs: 19
Dot balls so far: 8
Dot balls in last 2 overs: 3

Fours hit: 5
Sixes hit: 1

Powerplay completed: No

Remaining Batting Strength Score: 8.25

Final 1st innings score: 187
```

Example 2 — Middle-innings snapshot (after 12 overs, Section 6)

```
Match type: T20
Venue: Eden Gardens
Venue par score: 162
Batting team: India

Overs completed: 12
Balls remaining: 48
Runs scored: 97
Wickets lost: 3
Current run rate: 8.08

Runs in last 2 overs: 14
Dot balls so far: 21
```

```
Dot balls in last 2 overs: 4

Fours hit: 9
Sixes hit: 3

Powerplay completed: Yes

Remaining Batting Strength Score: 6.10

Final 1st innings score: 172
```

Example 3 — Death-overs snapshot (after 18 overs, Section 9)

```
Match type: T20
Venue: Melbourne Cricket Ground
Venue par score: 155
Batting team: Australia

Overs completed: 18
Balls remaining: 12
Runs scored: 163
Wickets lost: 5
Current run rate: 9.06

Runs in last 2 overs: 22
Dot balls so far: 29
Dot balls in last 2 overs: 2

Fours hit: 16
```

```
Sixes hit: 7

Powerplay completed: Yes

Remaining Batting Strength Score: 2.40

Final 1st innings score: 184
```

Chapter B

Player Classification Rules

B.1 Minimum Data Threshold

A player is assigned the label `INSUFFICIENT_DATA` and excluded from substantive classification if either of the following conditions holds:

- The player has appeared in fewer than **15 matches** in the dataset.
- The player has accumulated fewer than **250 total runs** *and* fewer than **10 total wickets**.

Players meeting the minimum threshold are then assessed by the ordered rule set in Section B.2. The first rule that matches determines the final classification; subsequent rules are not evaluated.

B.2 Ordered Classification Rules

Both ALL_ROUNDER sub-types receive the same `ALL_ROUNDER` label and the same RBSS weight (0.80). The distinction between bowling-dominant and batting-dominant all-rounders is made only for internal rule ordering and does not appear in the prompt.

B.3 RBSS Weight Design and Rationale

The RBSS weight for each classification was selected to reflect expected batting contribution in T20 cricket, not bowling quality. The weights are designed to be *stable*, *simple*,

Table B.1: Player classification rules applied in priority order

Classification	Conditions (all must hold)
BOWLER	Overs bowled per match ≥ 3 Economy rate ≤ 8.2 runs per over Bowling average ≤ 26 runs per wicket Batting average < 20 runs per dismissal
ALL_ROUNDER (bowling-dominant)	Overs bowled per match ≥ 2 Total wickets ≥ 25 Batting average ≥ 20 runs per dismissal Batting strike rate ≥ 120 Economy rate ≤ 8.8 runs per over
ALL_ROUNDER (batting-dominant)	Overs bowled per match ≥ 1 Total wickets ≥ 20 Batting average ≥ 25 runs per dismissal Batting strike rate ≥ 130
BATTER	Batting average ≥ 28 runs per dismissal Batting strike rate ≥ 125 Overs bowled per match < 0.5
UTILITY_PLAYER	All remaining players who meet the minimum data threshold and do not match any rule above.

rounded, and *consistent* across the full dataset, as language models interpret smooth and interpretable numeric values more reliably than arbitrary decimals.

B.3.1 Formula

The RBSS is computed at the end of each two-over section as:

$$\text{RBSS} = \sum_{p \in \mathcal{R}} w\big(\text{class}(p)\big) \tag{B.1}$$

where $\mathcal{R}$ is the set of players from the batting team's eleven who have not yet been dismissed at the end of that section, and $w(\cdot)$ maps each classification to its weight. The score is rounded to two decimal places.

B.3.2 Worked Example

Consider a match state at the end of over 12 in which the following players remain undismissed:

Table B.2: RBSS weights assigned to each player classification

Classification	Weight (w)	Design rationale
BATTER	1.00	Baseline reference unit. Specialist batters are the primary run-scoring resource; all other weights are calibrated relative to this value.
ALL_ROUNDER	0.80	T20 all-rounders typically bat at positions 5–7 with high strike rates but lower consistency than top-order batters. Bowling proficiency partially substitutes for batting depth, justifying a modest discount from 1.00.
UTILITY_PLAYER	0.65	Floaters and situational players with mixed skill sets and inconsistent output. Positioned between reliable batting contributors and tail-end bowlers.
INSUFFICIENT_DATA	0.45	Players with limited match history. A low weight (e.g. 0.30) would systematically undervalue unknown young batters; a high weight (e.g. 0.80) would be overly optimistic. The value 0.45 is a conservative mid-point that minimises expected error under uncertainty.
BOWLER	0.30	Specialist bowlers bat in the lower order (positions 8–11). The weight is non-zero to capture the occasional late-innings cameo hitting that is common in T20 cricket, but is kept low to reflect the generally limited batting expectation.

Player	Classification
Player A	BATTER
Player B	ALL_ROUNDER
Player C	ALL_ROUNDER
Player D	UTILITY_PLAYER
Player E	BOWLER
Player F	BOWLER
Player G	BOWLER

The RBSS at this point is:

$$\text{RBSS} = (1\times1.00)+(2\times0.80)+(1\times0.65)+(3\times0.30) = 1.00+1.60+0.65+0.90 = 4.15 \tag{B.2}$$

This is encoded in the natural-language prompt as `Remaining Batting Strength Score: 4.15`. The model can infer from this value that the batting side has limited remaining depth — a team of seven pure batters would yield an RBSS of 7.00, while seven pure bowlers would yield only 2.10.

B.3.3 Boundary Cases

A full eleven of `INSUFFICIENT_DATA` players at the start of an innings yields a maximum RBSS of $11 \times 0.45 = 4.95$. A team composed entirely of specialist batters would yield $11 \times 1.00 = 11.00$. In practice, RBSS values at the start of an innings typically fall in the range 6.0–8.5, decreasing as wickets fall and the lower-order bowlers approach the crease.

B.4 Implementation Reference

The classification logic is implemented in `forecaster/player_classification.py`, function `classify_t20i_player`. Player career statistics used as inputs to the classification rules are aggregated across all qualifying first-innings match rows in the dataset and stored in the SQLite table `first_innings_player_stats`.

Chapter C

Full Training Configuration

C.1 Quantisation Configuration (`BitsAndBytesConfig`)

Table C.1: 4-bit NF4 quantisation configuration

Parameter	Value
`load_in_4bit`	`True`
`bnb_4bit_quant_type`	`nf4`
`bnb_4bit_use_double_quant`	`True`
`bnb_4bit_compute_dtype`	`torch.float16`

C.2 LoRA Adapter Configuration (`LoraConfig`)

Table C.2: LoRA adapter configuration (LITE mode)

Parameter	Value
`r` (rank)	16
`lora_alpha`	32
`lora_dropout`	0.1
`bias`	`none`
`task_type`	`CAUSAL_LM`
`target_modules`	`q_proj, v_proj, k_proj, o_proj`

C.3 Training Arguments (`SFTConfig`)

C.4 Weights & Biases Run Details

C.5 Dataset and Model Artefact Identifiers

Table C.3: Full `SFTConfig` / `TrainingArguments` (LITE mode, Kaggle T4)

Parameter	Value
num_train_epochs	1
per_device_train_batch_size	4
gradient_accumulation_steps	2
per_device_eval_batch_size	4
eval_strategy	steps
eval_steps	100
save_strategy	steps
save_steps	200
save_total_limit	10
learning_rate	2×10^{-4}
lr_scheduler_type	cosine
warmup_ratio	0.03
weight_decay	0.001
max_grad_norm	0.3
max_seq_length	256
optim	paged_adamw_8bit
fp16	True
bf16	False
load_best_model_at_end	True
metric_for_best_model	eval_loss
greater_is_better	False
report_to	wandb
run_name	t20_score_prediction_keggle

Table C.4: Weights & Biases run metadata

Field	Value
Project	t20_score_prediction_keggle
Entity	ratnayakatilanka-university-of-sri-jayewardenepura
Run ID	v64wpa10
Run name	t20_score_prediction_keggle
Hardware	Kaggle NVIDIA T4 (16 GB VRAM)
Min eval/loss	1.856 (at step 800)
Best checkpoint SHA	00be3f3f2cb4af3cd727c68284ca3 dcbad7656a4

Table C.5: Hugging Face Hub artefact identifiers

Artefact	**Identifier**
Training/eval dataset (prompts)	`ratnayakatilanka/t20_score_prediction_prompts`
Evaluation dataset (summaries)	`ratnayakatilanka/t20_score_prediction_summary`
Fine-tuned LoRA adapter (Kaggle)	`ratnayakatilanka/t20_score_prediction_keggle-2026-04-18_04.16.40-lite`
Base model	`meta-llama/Llama-3.2-3B`

Chapter D

Sample Model Outputs

This appendix presents representative prediction examples drawn from the test split, illustrating the qualitative behaviour of each experimental condition. All examples shown are from actual test-set evaluations; no outputs have been fabricated or edited. The "Actual score" column gives the true final first-innings total from the match record.

D.1 Zero-Shot Llama 3.2-3B

The zero-shot baseline model frequently fails to produce a recognisable integer continuation. The following examples illustrate the range of output behaviours observed.

Table D.1: Representative zero-shot *Llama 3.2-3B* outputs

Match state	**Actual**	**Predicted**	**Raw model output**
India vs Australia, MCG, over 10, 84 runs/2 wkts	172	0	`The innings is still in progress...` (no integer extracted; fallback 0)
England vs NZ, Lord's, over 6, 58 runs/0 wkts	189	58	`58 runs` (regex captures 58 from the prompt echo; incorrect)
SA vs Pakistan, Lahore, over 16, 138 runs/4 wkts	161	161	`161` (correct; model coincidentally generates the right score)
Australia vs India, Sydney, over 4, 32 runs/1 wkt	168	0	`I cannot predict...` (model declines to answer; fallback 0)

The high rate of fallback-to-zero predictions and prompt-echo extractions is the primary driver of the extreme MAE (71.31 runs) and negative R^2 (-317.4%) for this condition.

D.2 Fine-Tuned Llama 3.2-3B (Step-800 Checkpoint)

After fine-tuning, the model consistently generates a standalone integer as its first token, reflecting successful learning of the output format. Prediction quality is markedly improved.

Table D.2: Representative fine-tuned *Llama 3.2-3B* outputs (step-800, 500 test datapoints)

Match state	**Actual**	**Predicted**	**Raw model output**
India vs Australia, MCG, over 10, 84 runs/2 wkts	172	168	`168` (error: -4 runs)
England vs NZ, Lord's, over 6, 58 runs/0 wkts	189	174	`174` (error: -15 runs)
SA vs Pakistan, Lahore, over 16, 138 runs/4 wkts	161	163	`163` (error: $+2$ runs)
Australia vs India, Sydney, over 4, 32 runs/1 wkt	168	142	`142` (error: -26 runs; early-innings high variance)

The early-innings prediction (over 4) shows the largest absolute error, which is expected: with only 4 overs completed, there is high inherent uncertainty in the final total. The model's prediction of 142 against an actual of 168 represents a reasonable range but is pulled towards the mean of the training distribution.

D.3 Zero-Shot GPT-4.1-nano

GPT-4.1-nano reliably produces a numeric output in response to the system instruction, eliminating the regex-fallback failures that affect the zero-shot Llama baseline. However, the predictions show a systematic underestimation bias in early- and middle-innings states. The powerplay and early-innings examples illustrate the characteristic underestimation pattern noted in Chapter 5: the frontier model applies a proportional scaling heuristic that does not account for the empirically observed late-innings acceleration in T20 scoring rates. Accuracy improves markedly in the death-overs state (over 16), where less extrap-

Table D.3: Representative zero-shot *GPT-4.1-nano* outputs (200 test datapoints)

Match state	Actual	Predicted	Raw model output
India vs Australia, MCG, over 10, 84 runs/2 wkts	172	140	`140` (error: −32 runs; underestimates acceleration)
England vs NZ, Lord's, over 6, 58 runs/0 wkts	189	116	`116` (error: −73 runs; severe underestimate in power-play)
SA vs Pakistan, Lahore, over 16, 138 runs/4 wkts	161	155	`155` (error: −6 runs; late-innings accuracy improves)
Australia vs India, Sydney, over 4, 32 runs/1 wkt	168	89	`89` (error: −79 runs; naive proportional projection)

olation is required.

D.4 Side-by-Side Comparison

Table D.4 directly compares the predictions of all three conditions on the same four test instances.

Table D.4: Side-by-side prediction comparison across all three conditions

Match state	Actual	Zero-shot Llama	Fine-tuned Llama	GPT-4.1-nano
India vs AUS, over 10, 84/2	172	0	168	140
ENG vs NZ, over 6, 58/0	189	58	174	116
SA vs PAK, over 16, 138/4	161	161	163	155
AUS vs IND, over 4, 32/1	168	0	142	89